

DATA STRUCTURES AND ALGORITHMS

23CSE203 LAB

MANUAL

SUBMITTED BY	
NAME	GUMMADI VISHNU
ROLL NUMBER	AV.SC.U4CSE25113
YEAR/SEM/SECTION	2ND YEAR /SEM-3/CSE-B
CREDITS	05

SUBMITTED TO	DR.RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFESSOR(SL GRADE)
DEPARTMENT	COMPUTING

DATA STRUCTURES & ALGORITHMS (DSA) LAB MANUAL

3rd Sem CSE - B | Academic Year 2026

S.NO	TITLE / TOPICS	DATE
1	**WEEK 1:** **Recursion** A) Recursive program for countdown from n to 1, followed by 'launch'. B) Recursive program to calculate P^n (compound interest growth factor P, years n). C) Recursive program to print first n terms of Fibonacci series. D) Recursive program to search for an employee ID in a list. E) Recursive program to calculate factorial (n!) for parcel arrangement.	14-07-26
2	**WEEK 2:** **Searching** A) Python program to perform Linear Search on given input. B) Python program to perform Binary Search on given input (verify with sorted and unsorted lists).	21-07-26
3	**WEEK 3:** **Elementary Sorting** A) Python program to implement Bubble Sort (user input). B) Python program to implement Selection Sort (user input). C) Python program to implement Insertion Sort (user input).	01-08-26
4	**WEEK 4:** **Advanced Sorting** A) Python program to implement Merge Sort (user input). B) Python program to implement Quick Sort (user input).	11-08-26
5	**WEEK 5:** **Linked Lists** Implement operations for: A) Singly Linked List (SLL) B) Doubly Linked List (DLL) C) Circular Linked List (CLL) Operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value 6. Delete Prior Node 7. Delete Last Node 8. Count Nodes 9. Display/Traverse 10. Exit	18-08-26
6	**WEEK 6:** **Stacks** A) Implement Stack operations (Push, Pop, Peek, Display) using Array. B) Implement Stack operations (Push, Pop, Peek, Display) using Linked List.	22-09-26
7	**WEEK 7:** **Queues** A) Implement Queue using Arrays. B) Implement Queue using Linked List. C) Implement Circular Queue using Arrays. D) Implement Circular Queue using Linked List.	29-09-26
8	**WEEK 8:** **Binary Trees** A) Implement Binary Tree with operations using Array. B) Implement Binary Tree with operations using Linked List.	06-10-26
9	**WEEK 9:** **Binary Search Trees** Python program to implement Binary Search Tree (BST) operations.	13-10-26
10	**WEEK 10:** **Write a python program to implement AVL trees with all its operations.**	27-10-26

11.	**WEEK 11:** **Hashing** Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).	27-10-26

Note:

The Lab External Exam is on November - 5th (Slot - 5, 6, 7).

Prerequisites:

1. Everyone should complete and submit the lab manual before the given date.
2. Kindly Bring 5 - 6 A4 Sheets.
3. Ensure you enter the lab exam with 75% attendance.

WEEK – 1

A.AIM:

A rocket launch system displays a count down before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a count down from n to 1, followed by launch.

INPUT CODE:

1.1.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/1.1.py (3.14.0)

File Edit Format Run Options Window Help

```
def countdown(n):
    if n <= 0:
        print("Launch")
        return
    print(n)
    countdown(n - 1)
n = int(input("Enter n: "))
countdown(n)
```

OUTPUT CASES:

```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/1.1.py =
Enter n: 10
10
9
8
7
6
5
4
3
2
1
Launch
>>>
```

EXPLANATION:

1.def countdown(n): — this snippet creates the recursive function and takes the starting number n as input.

2.if n <= 0: — this snippet is the base case. When the count has gone below 1, we stop going deeper.

3.print("Launch") — this snippet prints the final launch message once the countdown is over.

4.return — this snippet ends the current function call so recursion does not continue forever.

5.print(n) — this snippet shows the current number on the screen (5, then 4, then 3, and so on).

6.countdown(n - 1) — this snippet is the recursive call. It asks the same function to continue from the next smaller number.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Base case if n <= 0 removed	RecursionError	maximum recursion depth exceeded	**Kee**p Launch + return

B.AIM:

Banks calculate the growth of an investment using compound interest. The formula requires calculating powers which can be implemented recursively. Write a recursive program to calculate P^n[power].

INPUT:

2.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/2.py (3.14.0)

File Edit Format Run Options Window Help

```
def power(base, exp):
    if exp < 0:
        return 1 / power(base, -exp)
    if exp == 0:
        return 1
    return base * power(base, exp - 1)
print(power(23000,3))
print(power(2503,-1))
print(power(513,0))
```

OUTPUT CASES:

```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/2.py ====
12167000000000
0.00039952057530962844
1
>>>
```

EXPLANATION:

1.def power(base, exp): — this snippet defines the function that multiplies base by itself exp times.

2.if exp < 0: — this snippet handles a negative power (we need the reciprocal).

3.return 1 / power(base, -exp) — this snippet flips the exponent and divides 1 by that result (2^-3 → 0.125).

4.if exp == 0: — this snippet is the main base case. Any number to the power 0 is 1.

5.return 1 — this snippet stops recursion and sends 1 back to the previous call.

6.return base * power(base, exp - 1) — this snippet is the recursive step. It multiplies once and reduces the exponent by 1.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
if exp == 0: return 1 deleted	RecursionError	maximum recursion depth exceeded	**Keep power 0 = 1**
0 raised to a negative power	ZeroDivisionError	division by zero	**Treat 0^(-n) as undefined**

C.AIM:

An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

INPUT CODE:

3.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/3.py (3.14.0)

File Edit Format Run Options Window Help

```
def search_id(ids, target, index=0):
    if index >= len(ids):
        return -1
    if ids[index] == target:
        return index
    return search_id(ids, target, index + 1)
print(search_id([101, 205, 330, 412, 550, 678],330))
print(search_id([101, 205, 330, 412, 550, 678],990))
```

OUTPUT CASES:

```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/3.py
2
-1
>>>
```

EXPLANATION:

1.def search_id(ids, target, index=0): — this snippet starts the search. ids is the list, target is the ID we want, and index begins at 0.

2.index=0 — this snippet sets the starting position so the first call always looks at the first employee.

3.if index >= len(ids): — this snippet is the “not found” base case. If we have walked past the last ID, the list is finished.

4.return -1 — this snippet tells the caller that the employee ID does not exist in the list.

5.if ids[index] == target: — this snippet checks whether the current employee ID matches the one we are looking for.

6.return index — this snippet is the “found” base case. It sends back the position where the ID was found.

7.return search_id(ids, target, index + 1) — this snippet is the recursive call. It moves one step forward in the list and searches again.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Input Ravi instead of a number	ValueError	invalid literal for int()...	**IDs are digits only**
if index >= len(ids) missing	IndexError	list index out of range	**Stop and return -1**

D.AIM:

Write a python program to find n! -factorial of a number which can be used to find the number of possible ways to arrange in different parcels.

INPUT CODE:

4.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/4.py (3.14.0)

File Edit Format Run Options Window Help

```
def factorial(n):
    if n < 0:
        return None
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)
print(factorial(7))
print(factorial(0))
print(factorial(-1))
```

OUTPUT CASES:

```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/4.py ==
5040
1
None
>>>
```

EXPLANATION:

1.def factorial(n): — this snippet defines the function that finds n! (ways to arrange parcels).

2.If n < 0: — this snippet catches invalid input; factorial is not defined for negatives.

3.return None — this snippet signals “no answer” so the program can print an error.

4.if n == 0 or n == 1: — this snippet is the base case. Both 0! and 1! are 1.

5.return 1 — this snippet stops recursion and starts sending values back up.

6.return n * factorial(n - 1) — this snippet is the recursive step. It multiplies the current number by the factorial of the previous one.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Input -4	Logical / handled	Factorial is not defined for negative numbers	**if n < 0: return None**
Base case 0! / 1! removed	RecursionError	maximum recursion depth exceeded	**Keep return 1**
Input 4.5	ValueError	invalid literal for int()...	**Enter a whole number**

E.AIM:

Write a python program to print first n terms of a Fibonacci series.

INPUT CODE:

5.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/5.py (3.14.0)

File Edit Format Run Options Window Help

```
def fib(k):
    if k <= 0:
        return 0
    if k == 1:
        return 1
    return fib(k - 1) + fib(k - 2)

def print_fibonacci(n, i=0):
    if n <= 0:
        print("No terms to display")
        return
    if i == n:
        print()
        return
    print(fib(i), end=" ")
    print_fibonacci(n, i + 1)
print(print_fibonacci(7))
print(print_fibonacci(2))
print(print_fibonacci(-5))
```

OUTPUT CASES:

```
>>> ========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-1/5.py
0 1 1 2 3 5 8
None
0 1
None
No terms to display
None
>>>
```

EXPLANATION:

1.def fib(k): — this snippet finds one Fibonacci number at position k.

2.if k <= 0: return 0 — this snippet is the first base case. The 0th term is 0.

3.if k == 1: return 1 — this snippet is the second base case. The 1st term is 1.

4.return fib(k - 1) + fib(k - 2) — this snippet is the heart of Fibonacci. It adds the previous two terms.

5.def print_fibonacci(n, i=0): — this snippet prints the first n terms. i is which term we are on.

6.if n <= 0: — this snippet handles a bad request (0 or negative count).

7.print("No terms to display") — this snippet shows a clear message instead of an empty series.

8.print(fib(i), end=" ") — this snippet prints the current term on the same line with a space.

9.print_fibonacci(n, i + 1) — this snippet continues the series by moving to the next term.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
n = 0 or -5	Logical / handled	No terms to display	**if n <= 0 already handles it**
Base cases F(0), F(1) deleted	RecursionError	maximum recursion depth exceeded	**Keep 0 and 1 as stops**

WEEK – 2

A.AIM:

Write the Python program for linear search.

INPUT CODE:

1.py - C:\Users\vishn\Desktop\DSA PYTHON\WEEK-2\1.py (3.14.0)

File Edit Format Run Options Window Help

```
def linear_search(arr, key):
    i = 0
    n = len(arr)
    while i < n:
        if arr[i] == key:
            return i
        i = i + 1
    return -1
arr = [45, 12, 78, 23, 56, 89, 10]
print("List:", arr)
key = int(input("Enter element to search: "))
pos = linear_search(arr, key)
if pos != -1:
    print("Element", key, "found at position", pos)
else:
    print("Element", key, "not found")
```

OUTPUT CASES:

```
========== RESTART: C:\Users\vishn\Desktop\DSA PYTHON\WEEK-2\1.py ======
List: [45, 12, 78, 23, 56, 89, 10]
Enter element to search: 12
Element 12 found at position 1
>>>
========== RESTART: C:\Users\vishn\Desktop\DSA PYTHON\WEEK-2\1.py ======
List: [45, 12, 78, 23, 56, 89, 10]
Enter element to search: 88
Element 88 not found
>>>
```

EXPLANATION:

1.def linear_search(arr, key): — this snippet creates the function. arr is the list and key is the number we want.

2.i = 0 — this snippet starts the scan at the first position.

3.n = len(arr) — this snippet stores how many elements are in the list.

4.while i < n: — this snippet keeps checking until we have looked at every element.

5.if arr[i] == key: — this snippet compares the current element with the search key.

6.return i — this snippet stops the search and sends back the position where the key was found.

7.i = i + 1 — this snippet moves one step to the right when the current element is not the key.

8.return -1 — this snippet means we finished the list and the key is not there.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
User types letters (abc)	Runtime — ValueError	ValueError: invalid literal for int() with base 10: 'abc'	**Enter only numbers**
Loop condition written as while i <= n	Runtime — IndexError	IndexError: list index out of range	**Use while i < n**
Searching a value that is not in the list (99)	Logical (no crash)	Element 99 not found	**Already handled by return -1**

B.AIM:

Write the python code for the binary search with:

a) Sorted list of numbers

INPUT CODE:

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1


arr = [10, 20, 30, 40, 50, 60, 70]
print("Sorted list:", arr)
key = int(input("Enter element to search: "))
pos = binary_search(arr, key)
if pos != -1:
    print("Element", key, "found at position", pos)
else:
    print("Element", key, "not found")
```

OUTPUT CASES:

```
============ RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-2/2.py ==
Sorted list: [10, 20, 30, 40, 50, 60, 70]
Enter element to search: 20
Element 20 found at position 1
>>>
============ RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-2/2.py ==
Sorted list: [10, 20, 30, 40, 50, 60, 70]
Enter element to search: 80
Element 80 not found
>>>
```

EXPLANATION:

1.def binary_search(arr, key): — this snippet defines binary search. The list arr must already be sorted.

2.low = 0 — this snippet marks the left end of the current search range.

3.high = len(arr) - 1 — this snippet marks the right end of the current search range.

4.while low <= high: — this snippet continues as long as there is still a valid range to search.

5.mid = (low + high) // 2 — this snippet finds the middle index (integer division, no decimal).

6.if arr[mid] == key: — this snippet checks whether the middle element is the one we want.

7.return mid — this snippet is the success stop. The key is at mid.

8.elif arr[mid] < key: — this snippet means the key is bigger, so it must be on the right side.

9.low = mid + 1 — this snippet throws away the left half, including mid.

10.else: high = mid - 1 — this snippet means the key is smaller, so we throw away the right half.

11.return -1 — this snippet means the range became empty and the key is not in the list.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
User types text (forty)	Runtime — ValueError	ValueError: invalid literal for int()...	**Enter a number**
while low < high instead of low <= high	Logical	Key exists but program says **not found**	**Use while low <= high**
mid = (low + high) / 2 (single slash)	Runtime / logical	TypeError: list indices must be integers	**Use integer division //**

B.AIM:

b) Unsorted list of numbers.

Write the Python program for linear search.

INPUT CODE:

2.2 un sorted.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-2/2.2 un sorted.py (3.14.0)

File Edit Format Run Options Window Help

```
def binary_search(arr, key):
    low = 0
    high = len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1


def binary_search_unsorted(arr, key):
    sorted_arr = sorted(arr)          # sort first
    pos = binary_search(sorted_arr, key)
    return sorted_arr, pos


arr = [45, 12, 78, 23, 56, 89, 10]
print("Original (unsorted) list:", arr)
key = int(input("Enter element to search: "))
sorted_arr, pos = binary_search_unsorted(arr, key)
print("After sorting:", sorted_arr)
if pos != -1:
    print("Element", key, "found at position", pos, "in the sorted list")
else:
    print("Element", key, "not found")
```

OUTPUT CASES:

```
====== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-2/2.2 un sorted.py ==
Original (unsorted) list: [45, 12, 78, 23, 56, 89, 10]
Enter element to search: 78
After sorting: [10, 12, 23, 45, 56, 78, 89]
Element 78 found at position 5 in the sorted list
>>>
====== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-2/2.2 un sorted.py ==
Original (unsorted) list: [45, 12, 78, 23, 56, 89, 10]
Enter element to search: 99
After sorting: [10, 12, 23, 45, 56, 78, 89]
Element 99 not found
```

EXPLANATION:

1.sorted_arr = sorted(arr) — this snippet makes an ascending copy. Binary search is safe only after this.

2.pos = binary_search(sorted_arr, key) — this snippet runs the same binary search on the sorted copy.

3.return sorted_arr, pos — this snippet sends back both the sorted list (so we can print it) and the index.

4.print("After sorting:", sorted_arr) — this snippet shows that we sorted before searching.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Empty list []	Logical (no crash)	not found	**Correct for an empty list**

WEEK – 3

A.AIM:

Write the Python program to implement bubble sort where the elements are taken from the user.

INPUT CODE:

1.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-3/1.py (3.14.0)

File Edit Format Run Options Window Help

```
def bubble_sort(arr):
    n = len(arr)
    i = 0
    while i < n - 1:
        j = 0
        swapped = False
        while j < n - i - 1:
            if arr[j] > arr[j + 1]:
                temp = arr[j]
                arr[j] = arr[j + 1]
                arr[j + 1] = temp
                swapped = True
            j = j + 1
        if not swapped:
            break
        i = i + 1
    return arr


n = int(input("Enter number of elements: "))
print("Enter", n, "elements (space separated):")
arr = list(map(int, input().split()))
print("Before sorting:", arr)
print("After bubble sort:", bubble_sort(arr))
```

OUTPUT CASES:

```
========== RESTART: C:\Users\vishn\Desktop\DSA PYTHON\WEEK-3\1.py ==========
Enter number of elements: 4
Enter 4 elements (space separated):
3 5 7 9
Before sorting: [3, 5, 7, 9]
After bubble sort: [3, 5, 7, 9]
>>>
========== RESTART: C:\Users\vishn\Desktop\DSA PYTHON\WEEK-3\1.py ==========
Enter number of elements: 5
Enter 5 elements (space separated):
23 34 632 2 1
Before sorting: [23, 34, 632, 2, 1]
After bubble sort: [1, 2, 23, 34, 632]
>>>
========== RESTART: C:\Users\vishn\Desktop\DSA PYTHON\WEEK-3\1.py ==========
Enter number of elements: 5
Enter 5 elements (space separated):
22 1 54 43
Before sorting: [22, 1, 54, 43]
After bubble sort: [1, 22, 43, 54]
>>>
```

EXPLANATION:

1.n = int(input("Enter number of elements: ")) — this snippet asks how many numbers the user wants to sort.

2.arr = list(map(int, input().split())) — this snippet reads the numbers from one line and turns them into a list of integers.

3.i = 0 / while i < n - 1: — this snippet is the outer pass. After pass i, the last i elements are already in place.

4.j = 0 / while j < n - i - 1: — this snippet walks through the unsorted part, comparing neighbours.

5.if arr[j] > arr[j + 1]: — this snippet checks whether two neighbours are in the wrong order.

6.temp = arr[j] … arr[j + 1] = temp — this snippet swaps the two neighbours.

7.swapped = True — this snippet remembers that at least one swap happened in this pass.

8.if not swapped: break — this snippet stops early if the list is already sorted (no swaps).

9.return arr — this snippet sends the sorted list back to be printed.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
n typed as text (five)	Runtime — ValueError	ValueError: invalid literal for int() with base 10: 'five'	**Enter a whole number for n**
Inner loop written as while j < n	Runtime — IndexError	IndexError: list index out of range	**Use while j < n - i - 1**
n = 0 or n = -4	Logical / handled	Number of elements must be a positive integer	**Enter n ≥ 1**

B.AIM:

Write the Python program to implement insertion sort where the elements are taken from the user.

INPUT CODE:

2.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-3/2.py (3.14.0)

File Edit Format Run Options Window Help

```
def insertion_sort(arr):
    n = len(arr)
    i = 1
    while i < n:
        key = arr[i]
        j = i - 1
        while j >= 0 and arr[j] > key:
            arr[j + 1] = arr[j]
            j = j - 1
        arr[j + 1] = key
        i = i + 1
    return arr


n = int(input("Enter number of elements: "))
print("Enter", n, "elements (space separated):")
arr = list(map(int, input().split()))
print("Before sorting:", arr)
print("After insertion sort:", insertion_sort(arr))
```

OUTPUT CASES:

```
Python 3.14.0 (tags/v3.14.0:ebf955d, Oct  7 2025, 10:15:03) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
============ RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-3/2.py ============
Enter number of elements: 6
Enter 6 elements (space separated):
23 535 62 4 54 234
Before sorting: [23, 535, 62, 4, 54, 234]
After insertion sort: [4, 23, 54, 62, 234, 535]
>>>
```

EXPLANATION:

1.i = 1 / while i < n: — this snippet starts from the second element. The first element is already a sorted list of size 1.

2.key = arr[i] — this snippet picks the next value that must be inserted into the left (sorted) part.

3.j = i - 1 — this snippet starts comparing key with the element just to its left.

4.while j >= 0 and arr[j] > key: — this snippet shifts larger elements one step right until the hole for key is found.

5.arr[j + 1] = arr[j] — this snippet moves a bigger element to the right to make space.

6.j = j - 1 — this snippet walks further left.

7.arr[j + 1] = key — this snippet drops key into the correct place.

8.i = i + 1 — this snippet moves on to the next unsorted element.

ERROR TABLE:

ERROR	**TYPE**	**DISPLAYED AS**	**FIX**
A list value is not a number (1 2 a 4)	Runtime — ValueError	ValueError: invalid literal for int() with base 10: 'a'	**Enter only integers**
Inner condition is only arr[j] > key (no j >= 0)	Runtime — IndexError	IndexError: list index out of range	**Keep while j >= 0 and arr[j] > key**
Outer loop starts at i = 0 instead of i = 1	Logical	----------	**Start from the second element (i = 1)**

C.AIM:

Write the Python program to implement selection sort where the elements are taken from the user.

INPUT CODE:

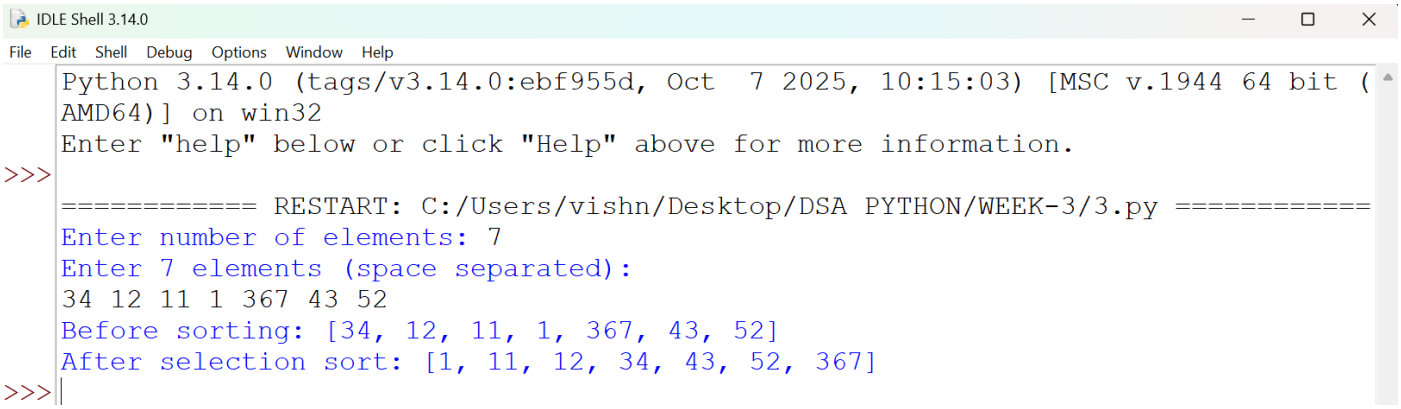
3.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-3/3.py (3.14.0)

File Edit Format Run Options Window Help

```
def selection_sort(arr):
    n = len(arr)
    i = 0
    while i < n - 1:
        min_index = i
        j = i + 1
        while j < n:
            if arr[j] < arr[min_index]:
                min_index = j
            j = j + 1
        temp = arr[i]
        arr[i] = arr[min_index]
        arr[min_index] = temp
        i = i + 1
    return arr


n = int(input("Enter number of elements: "))
print("Enter", n, "elements (space separated):")
arr = list(map(int, input().split()))
print("Before sorting:", arr)
print("After selection sort:", selection_sort(arr))
```

OUTPUT CASES:



EXPLANATION:

1.i = 0 / while i < n - 1: — this snippet marks the position that should receive the next smallest value.

2.min_index = i — this snippet assumes the current position holds the smallest so far.

3.j = i + 1 / while j < n: — this snippet scans the rest of the list to find anything smaller.

4.if arr[j] < arr[min_index]: — this snippet updates the record when a smaller value is found.

5.min_index = j — this snippet remembers where that smaller value lives.

6.temp = arr[i] … arr[min_index] = temp — this snippet swaps the smallest remaining value into position i.

7.i = i + 1 — this snippet locks that position and continues with the rest of the list.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Too few numbers entered (n=4 but only 10 20)	Logical / handled	You entered 2 values, expected 4	**Type exactly n numbers**
Forgot to reset min_index = i each pass	Logical	List is not fully sorted	**Set min_index = i at the start of every outer pass**
Compared with > instead of <	Logical	List comes out in descending order	**Use if arr[j] < arr[min_index] for ascending**

WEEK – 4

A.AIM:

Write the Python program to implement merge sort where the elements are taken from the user.

INPUT CODE:

1.py - C:/Users/Lab/Desktop/CSE25113/WEEK-4/1.py (3.11.4)

File Edit Format Run Options Window Help

```
def mergeSort(arr): #the inital main function
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    leftHalf = arr[:mid]
    rightHalf = arr[mid:]

    sortedLeft = mergeSort(leftHalf)
    sortedRight = mergeSort(rightHalf)

    return merge(sortedLeft, sortedRight)
def merge(left, right):#the sub function
    result = []
    i = j = 0

    while i < len(left) and j < len(right):
        if left[i] < right[j]:
            result.append(left[i])#
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])

    return result
mylist = [3, 7, 6, -10, 15, 23.5, 55, -13]
mysortedlist = mergeSort(mylist)
print("Sorted array:", mysortedlist)
print("G.VISHNU CSE25113")
```

OUTPUT CASES:

```
= RESTART: C:/Users/Lab/Desktop/CSE25113/WEEK-4/1.py
Sorted array: [-13, -10, 3, 6, 7, 15, 23.5, 55]
G.VISHNU CSE25113
>>>
```

EXPLANATION:

1.mid = len(arr) // 2 — this snippet finds the middle position of the array to divide it into two halves.

2.leftHalf = arr[:mid] — this snippet creates the left half of the array using slicing.

3.rightHalf = arr[mid:] — this snippet creates the right half of the array using slicing.

4.sortedLeft = mergeSort(leftHalf) — this snippet recursively calls the mergeSort function to sort the left half.

5.sortedRight = mergeSort(rightHalf) — this snippet recursively calls the mergeSort function to sort the right half.

6.return merge(sortedLeft, sortedRight) — this snippet executes the user-defined merge function to combine the two sorted halves.

7.while i < len(left) and j < len(right): — this snippet repeatedly compares elements from both halves until one half has no more elements.

8.result.extend(left[i:]) — this snippet adds all remaining elements from the left half to the result after the comparison loop ends.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Forgot the base case if len(arr) <= 1	Logical	RecursionError: maximum recursion depth exceeded	Add if len(arr) <= 1: return arr
Used > instead of < in if left[i] < right[j]	Logical	List is sorted in descending order	Use if left[i] < right[j]: for ascending order
Forgot to add the remaining elements after the while loo	Logical	Some elements are missing from the sorted array	Use result.extend(left[i:]) and result.extend(right[j:])

B.AIM:

Write Python program to implement quick sort where the elements are taken from the user.

INPUT CODE:

2.py - C:/Users/Lab/Desktop/CSE25113/WEEK-4/2.py (3.11.4)

File Edit Format Run Options Window Help

```
def partition(array, low, high):
  pivot = array[high]
  i = low - 1

  for j in range(low, high):
     if array[j] <= pivot:
       i += 1
       array[i], array[j] = array[j], array[i]

  array[i+1], array[high] = array[high], array[i+1]
  return i+1

def quicksort(array, low=0, high=None):
  if high is None:
    high = len(array) - 1

  if low < high:
    pivot_index = partition(array, low, high)
    quicksort(array, low, pivot_index-1)
    quicksort(array, pivot_index+1, high)

# Ask the user to enter numbers separated by spaces
mylist = list(map(int, input("Enter numbers separated by spaces: ").split()))

quicksort(mylist)
print("Sorted array:", mylist)
print("G.VISHNU CSE25113")
```

OUTPUT PAGES:

```
========== RESTART: C:/Users/Lab/Desktop/CSE25113/WEEK-4/2.py ==
Enter numbers separated by spaces: 23 54 32 35 26 7
Sorted array: [7, 23, 26, 32, 35, 54]
G.VISHNU CSE25113
>>>
```

EXPLANATION:

1.pivot = array[high] — this snippet selects the last element of the array as the pivot for partitioning.
2.i = low - 1 — this snippet initializes the position where the next element smaller than or equal to the pivot will be placed.
3.for j in range(low, high): — this snippet checks each element from the lower boundary up to the element before the pivot.
4.if array[j] <= pivot: — this snippet checks whether the current element should be placed on the left side of the pivot.
5.array[i], array[j] = array[j], array[i] — this snippet swaps elements so that values smaller than or equal to the pivot are moved to the correct side.
6.array[i+1], array[high] = array[high], array[i+1] — this snippet places the pivot in its correct sorted position.

7.return i+1 — this snippet returns the final position of the pivot after partitioning.
8.pivot_index = partition(array, low, high) — this snippet calls the partition function and stores the pivot's correct position.
9.quicksort(array, low, pivot_index-1) — this snippet recursively sorts the elements on the left side of the pivot.
10.quicksort(array, pivot_index+1, high) — this snippet recursively sorts the elements on the right side of the pivot.
11.quicksort(mylist) — this snippet starts the Quick Sort algorithm on the given list.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Forgot high = len(array) - 1 when high is None	Logical	Quicksort does not process the complete array	Set high = len(array) - 1 when high is None
Used array[high-1] instead of array[high] for the pivot	Logical	List is not sorted correctly	Use pivot = array[high]
Used incorrect recursive ranges after partitioning	Logical	List is partially sorted or incorrectly sorted	**Use quicksort(array, low, pivot_index-1) and quicksort(array, pivot_index+1, high)**

WEEK – 5

A.AIM:

Implement a simple linked list and perform the operations: create, insert at beginning, insert at end, insert at index, delete by value, delete first node, delete last node, count number of nodes, display or traverse, exit.

INPUT CODE :

5.1.py - C:/Users/Lab/Desktop/CSE25113/WEEK-5/5.1.py (3.11.4)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter number of nodes: "))

        if n <= 0:
            print("Number of nodes must be greater than 0")
            return

        for i in range(n):
            data = int(input("Enter element: "))
            new_node = Node(data)

            if self.head is None:
                self.head = new_node
            else:
                temp = self.head

                while temp.next is not None:
                    temp = temp.next

                temp.next = new_node

        print("Linked list created successfully")

    def insert_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

        print(data, "inserted at beginning")

    def insert_end(self, data):
        new_node = Node(data)

```

5.1.py - C:/Users/Lab/Desktop/CSE25113/WEEK-5/5.1.py (3.11.4)

File Edit Format Run Options Window Help

```
43
44          if self.head is None:
45              self.head = new_node
46          else:
47              temp = self.head
48
49              while temp.next is not None:
50                  temp = temp.next
51
52              temp.next = new_node
53
54          print(data, "inserted at end")
55
56      def insert_index(self, data, index):
57          if index < 0:
58              print("Invalid index")
59              return
60
61          if index == 0:
62              self.insert_beginning(data)
63              return
64
65          temp = self.head
66
67          for i in range(index - 1):
68              if temp is None:
69                  print("Index out of range")
70                  return
71
72              temp = temp.next
73
74          if temp is None:
75              print("Index out of range")
76              return
77
78          new_node = Node(data)
79          new_node.next = temp.next
80          temp.next = new_node
81
82          print(data, "inserted at index", index)
83
```

5.1.py - C:/Users/Lab/Desktop/CSE25113/WEEK-5/5.1.py (3.11.4)

File Edit Format Run Options Window Help

```
83
84      def delete_value(self, value):
85          if self.head is None:
86              print("Linked list is empty")
87              return
88
89          if self.head.data == value:
90              self.head = self.head.next
91              print(value, "deleted from the list")
92              return
93
94          temp = self.head
95
96          while temp.next is not None:
97              if temp.next.data == value:
98                  temp.next = temp.next.next
99                  print(value, "deleted from the list")
100                 return
101
102             temp = temp.next
103
104         print(value, "not found in the list")
105
106     def delete_first(self):
107         if self.head is None:
108             print("Linked list is empty")
109             return
110
111         value = self.head.data
112         self.head = self.head.next
113
114         print(value, "deleted from beginning")
115
116     def delete_last(self):
117         if self.head is None:
118             print("Linked list is empty")
119             return
120
121         if self.head.next is None:
122             value = self.head.data
123             self.head = None
124             print(value, "deleted from end")
125             return
```

5.1.py - C:/Users/Lab/Desktop/CSE25113/WEEK-5/5.1.py (3.11.4)

File Edit Format Run Options Window Help

```
126
127         temp = self.head
128
129         while temp.next.next is not None:
130             temp = temp.next
131
132         value = temp.next.data
133         temp.next = None
134
135         print(value, "deleted from end")
136
137     def count(self):
138         count = 0
139         temp = self.head
140
141         while temp is not None:
142             count += 1
143             temp = temp.next
144
145         print("Number of nodes:", count)
146
147     def display(self):
148         if self.head is None:
149             print("Linked list is empty")
150             return
151
152         temp = self.head
153
154         print("Linked list:")
155
156         while temp is not None:
157             print(temp.data, end=" -> ")
158             temp = temp.next
159
160         print("None")
161
162
163 ll = LinkedList()
```

```
while True:
    print("\n--- LINKED LIST MENU ---")
    print("1. Create")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        ll.create()

    elif choice == 2:
        data = int(input("Enter element: "))
        ll.insert_beginning(data)

    elif choice == 3:
        data = int(input("Enter element: "))
        ll.insert_end(data)

    elif choice == 4:
        data = int(input("Enter element: "))
        index = int(input("Enter index: "))
        ll.insert_index(data, index)

    elif choice == 5:
        value = int(input("Enter value to delete: "))
        ll.delete_value(value)

    elif choice == 6:
        ll.delete_first()

    elif choice == 7:
        ll.delete_last()

    elif choice == 8:
        ll.count()

    elif choice == 9:
        ll.display()

    elif choice == 10:
        print("Program terminated")
        break

    else:
        print("Invalid choice")

```

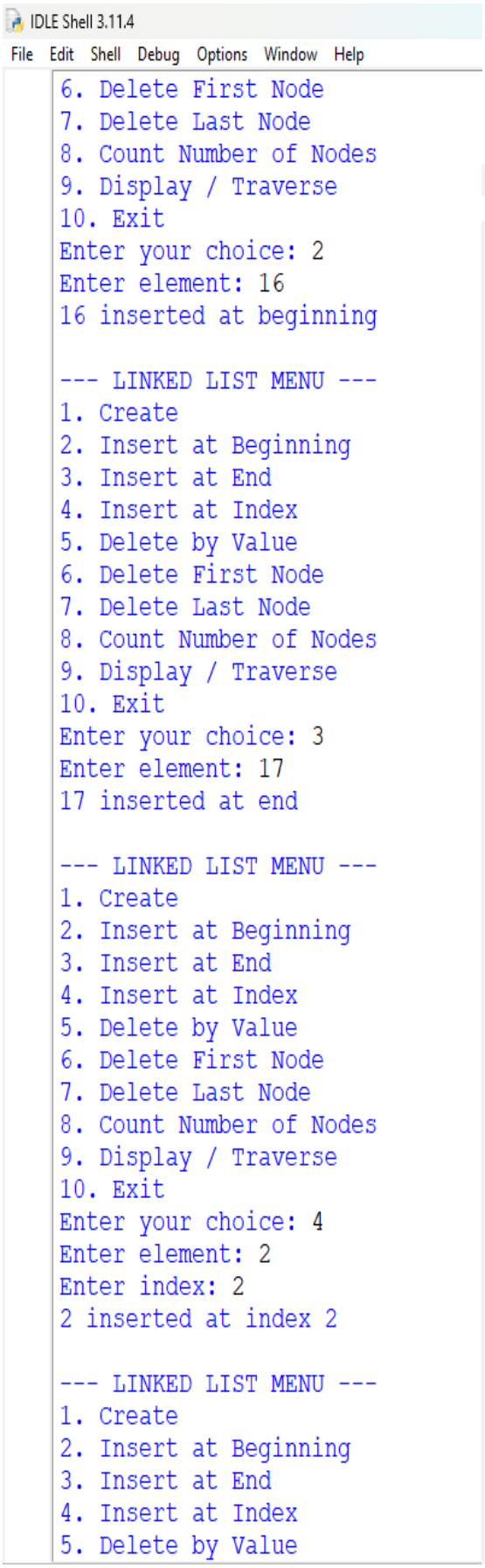
OUTPUT :

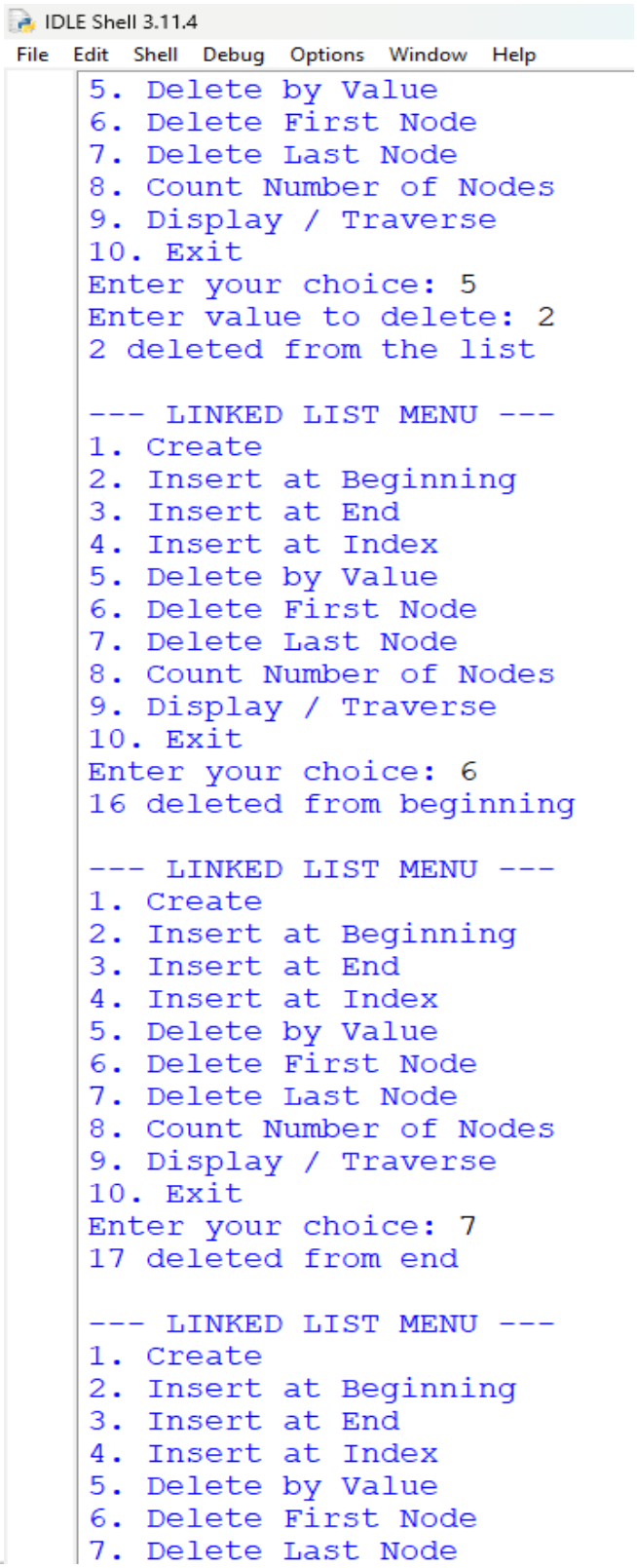
```
>>>
= RESTART: C:/Users/Lab/Desktop/CSE25113/WEEK-5/5.1.py

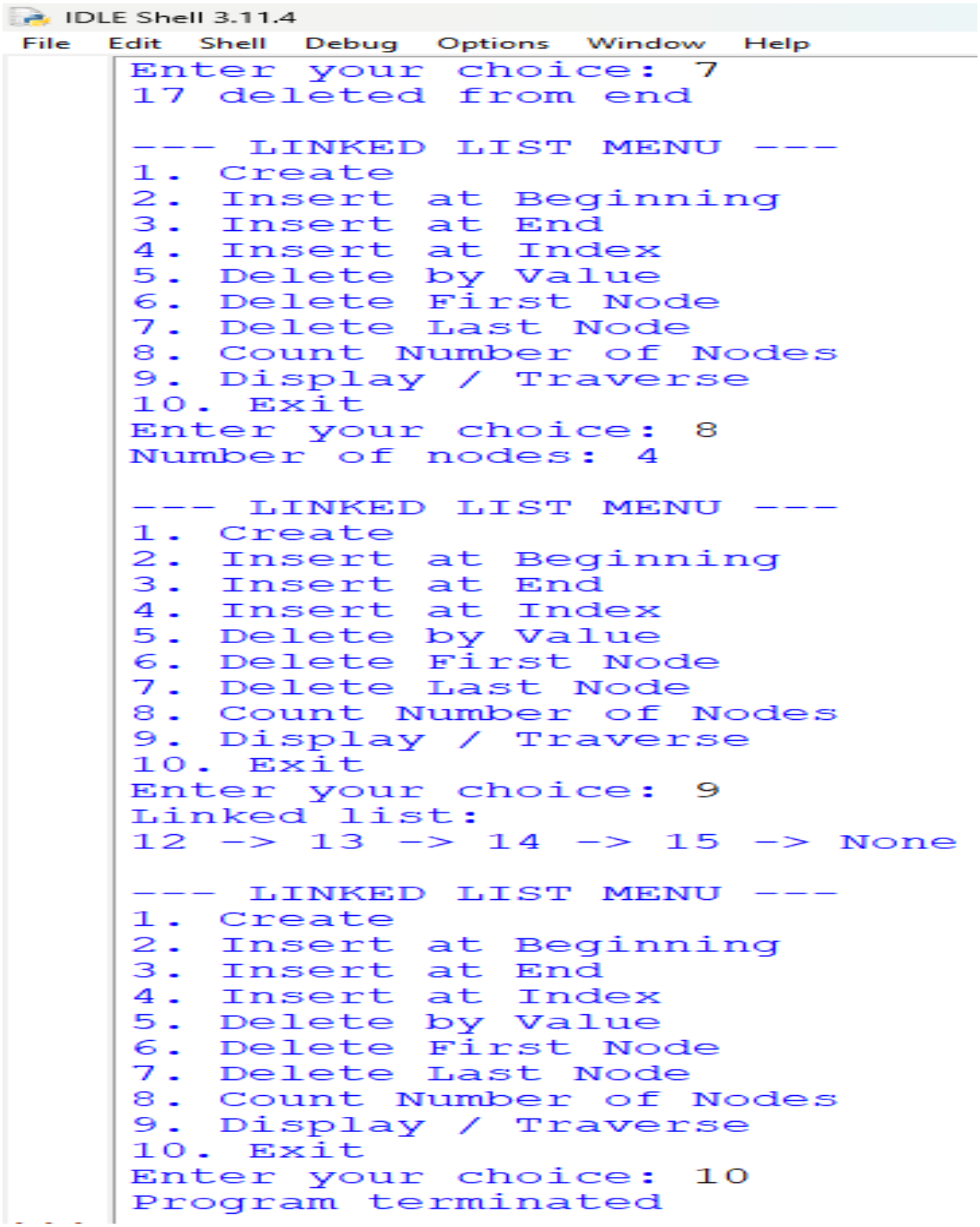
--- LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 4
Enter element: 12
Enter element: 13
Enter element: 14
Enter element: 15
Linked list created successfully

--- LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter element: 16
16 inserted at beginning

--- LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
```


IDLE Shell 3.11.4
File Edit Shell Debug Options Window Help
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter element: 16
16 inserted at beginning
--- LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter element: 17
17 inserted at end
--- LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter element: 2
Enter index: 2
2 inserted at index 2
--- LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value


IDLE Shell 3.11.4
File Edit Shell Debug Options Window Help
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 2
2 deleted from the list
--- LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
16 deleted from beginning
--- LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
17 deleted from end
--- LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node



EXPLANATION:

1. self.data = data / self.next = None — this snippet makes a node that holds a value and a link to the next node. None means there is no next node yet.

2.self.head = None — this snippet starts the linked list empty, with the head pointing to nothing.

3.newNode.next = self.head / self.head = newNode — this snippet links the new node to the old first node, then makes the new node the head.

4.while current.next: current = current.next — this snippet walks through the list until it reaches the last node, the one whose next is None.

5.while current is not None and position + 1 != index: — this snippet moves forward until it reaches the node just before the given index.

6.newNode.next = current.next / current.next = newNode — this snippet places the new node between two existing nodes without breaking the chain.

7.prev.next = current.next — this snippet skips over the node that holds the value, which removes it from the list.

8.self.head = self.head.next — this snippet deletes the first node by moving the head to the second node.

9.while current.next.next: … current.next = None — this snippet stops at the second-last node and cuts its link, which deletes the last node.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Deleted from an empty list without the if self.head is None check	Runtime — AttributeError	AttributeError: 'NoneType' object has no attribute 'next'	Add if self.head is None: print ("List is empty") before deleting
Wrote self.head = newNode before newNode.next = self.head in insert at beginning	Logical	Display prints the same value again and again without stopping (the node points to itself)	Set newNode.next = self.head first, then self.head = newNode
Called insertAtEnd(value) without self. inside create	Runtime — NameError	NameError: name 'insertAtEnd' is not defined	**Call it as self.insertAtEnd(value)**

B.AIM:

To write a menu-driven Python program that implements a doubly linked list and performs these operations: create, insert at beginning, insert at end, insert at index, delete by value, delete first node, delete last node, count nodes, display (traverse) and exit.

INPUT CODE:

2.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-5/2.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None
class DoublyLinkedList:
    def __init__(self):
        self.head = None
    def create(self):
        n = int(input("Enter number of nodes: "))
        if n <= 0:
            print("Number of nodes must be greater than 0")
            return
        for i in range(n):
            data = int(input("Enter element: "))
            new_node = Node(data)
            if self.head is None:
                self.head = new_node
            else:
                temp = self.head
                while temp.next is not None:
                    temp = temp.next
                temp.next = new_node
                new_node.prev = temp
        print("Doubly linked list created successfully")
    def insert_beginning(self, data):
        new_node = Node(data)
        if self.head is not None:
            new_node.next = self.head
            self.head.prev = new_node
        self.head = new_node
        print(data, "inserted at beginning")
    def insert_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next is not None:
```

2.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-5/2.py (3.14.0)

File Edit Format Run Options Window Help

```
            while temp.next is not None:
                temp = temp.next
            temp.next = new_node
            new_node.prev = temp
        print(data, "inserted at end")
    def insert_index(self, data, index):
        if index < 0:
            print("Invalid index")
            return
        if index == 0:
            self.insert_beginning(data)
            return
        temp = self.head
        for i in range(index - 1):
            if temp is None:
                print("Index out of range")
                return
            temp = temp.next
        if temp is None:
            print("Index out of range")
            return
        new_node = Node(data)
        new_node.next = temp.next
        new_node.prev = temp
        if temp.next is not None:
            temp.next.prev = new_node
        temp.next = new_node
        print(data, "inserted at index", index)
    def delete_value(self, value):
        if self.head is None:
            print("Linked list is empty")
            return
        temp = self.head
        while temp is not None and temp.data != value:
            temp = temp.next
        if temp is None:
            print(value, "not found in the list")
            return
        if temp.prev is not None:
```

2.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-5/2.py (3.14.0)

File Edit Format Run Options Window Help

```
            temp.prev.next = temp.next
        else:
            self.head = temp.next
        if temp.next is not None:
            temp.next.prev = temp.prev
        print(value, "deleted from the list")
    def delete_first(self):
        if self.head is None:
            print("Linked list is empty")
            return
        value = self.head.data
        self.head = self.head.next
        if self.head is not None:
            self.head.prev = None
        print(value, "deleted from beginning")
    def delete_last(self):
        if self.head is None:
            print("Linked list is empty")
            return
        temp = self.head
        while temp.next is not None:
            temp = temp.next
        value = temp.data
        if temp.prev is None:
            self.head = None
        else:
            temp.prev.next = None
        print(value, "deleted from end")
    def count(self):
        count = 0
        temp = self.head
        while temp is not None:
            count += 1
            temp = temp.next
        print("Number of nodes:", count)
    def display(self):
        if self.head is None:
            print("Linked list is empty")
            return
```

2.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-5/2.py (3.14.0)

File Edit Format Run Options Window Help

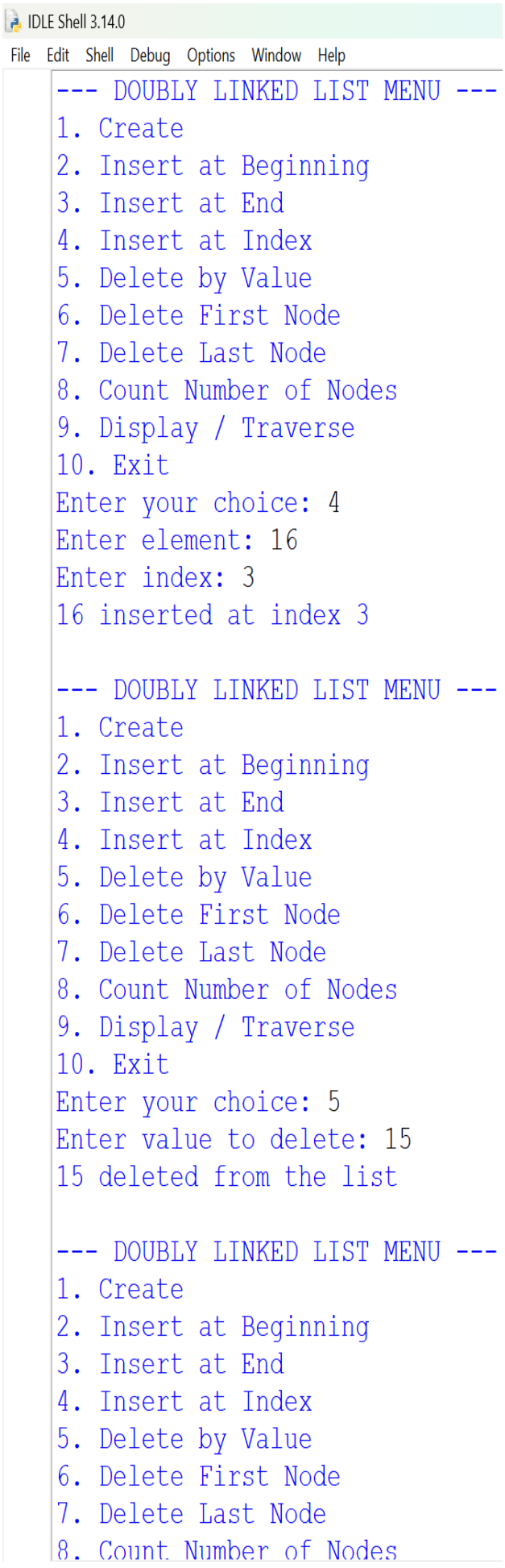
```
            return
        temp = self.head
        print("Doubly linked list:")
        print("None", end=" <-> ")
        while temp is not None:
            print(temp.data, end=" <-> ")
            temp = temp.next
        print("None")
dll = DoublyLinkedList()
while True:
    print("\n--- DOUBLY LINKED LIST MENU ---")
    print("1. Create")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        dll.create()
    elif choice == 2:
        data = int(input("Enter element: "))
        dll.insert_beginning(data)
    elif choice == 3:
        data = int(input("Enter element: "))
        dll.insert_end(data)
    elif choice == 4:
        data = int(input("Enter element: "))
        index = int(input("Enter index: "))
        dll.insert_index(data, index)
    elif choice == 5:
        value = int(input("Enter value to delete: "))
        dll.delete_value(value)
    elif choice == 6:
        dll.delete_first()
    elif choice == 7:
        dll.delete_last()
    elif choice == 8:
        dll.count()
    elif choice == 9:
        dll.display()
    elif choice == 10:
        print("Program terminated")
        break
    else:
        print("Invalid choice")
```

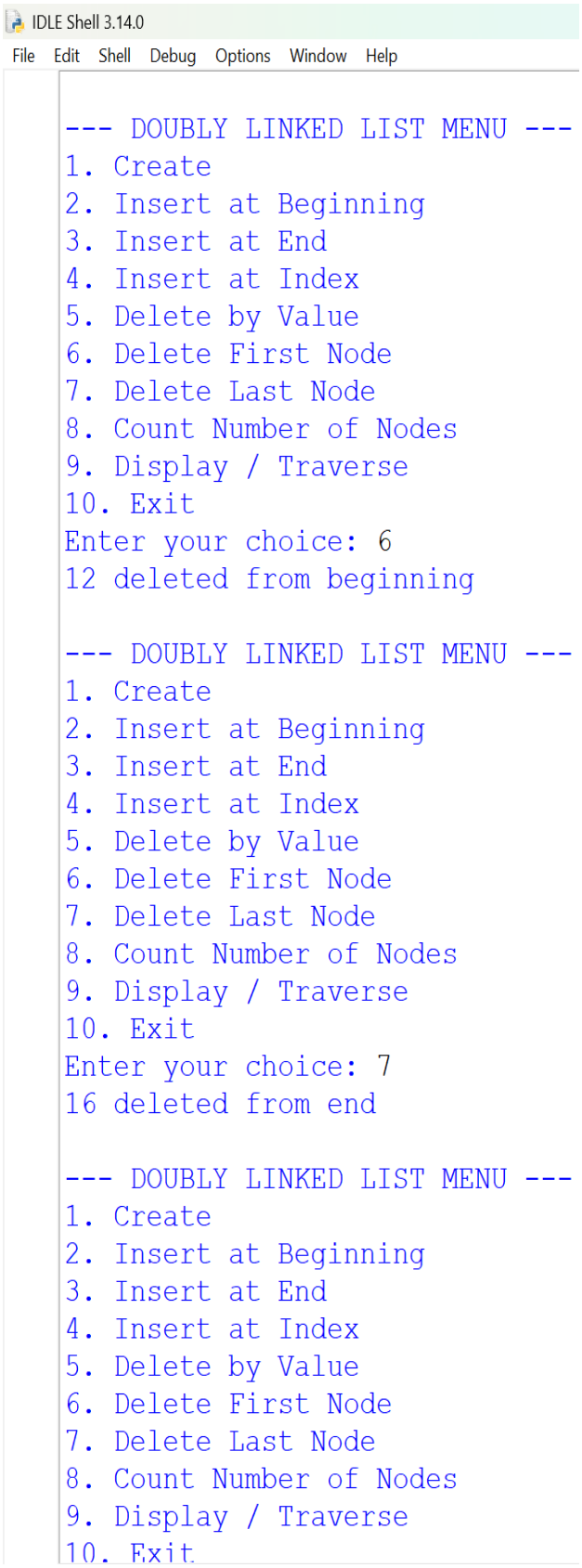
OUTPUT:

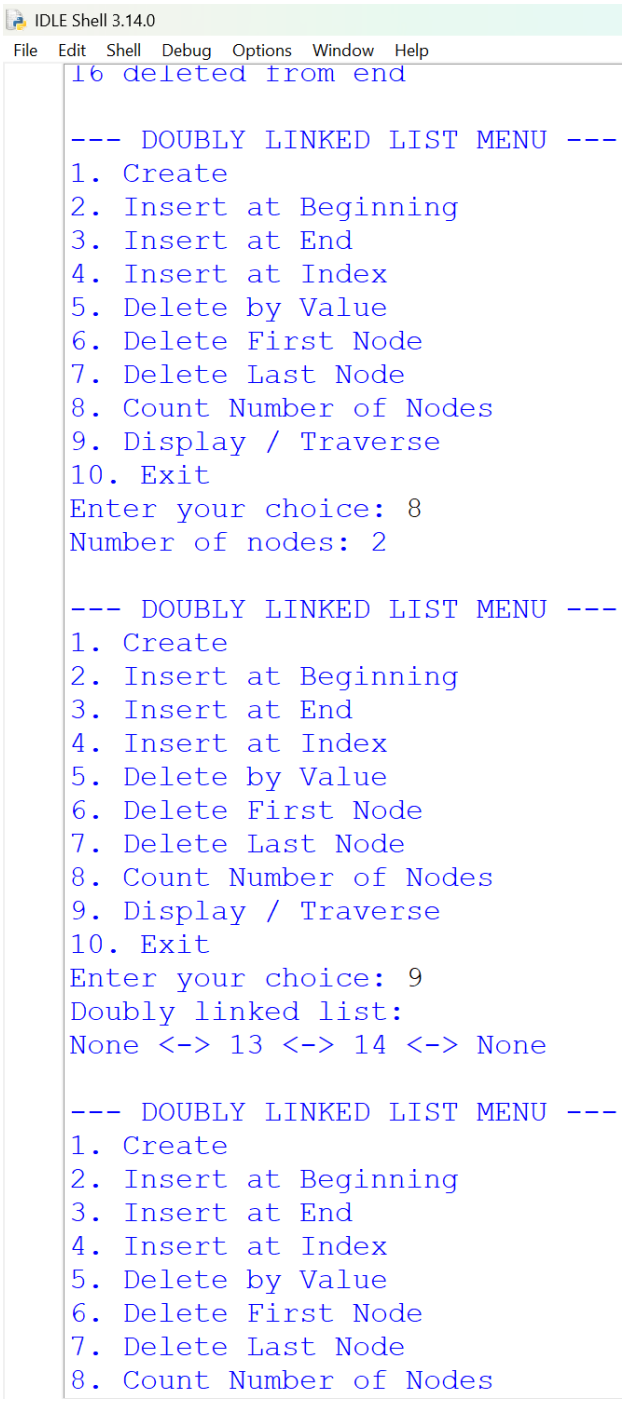
```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-5/2.py

--- DOUBLY LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter element: 12
Enter element: 13
Enter element: 14
Doubly linked list created successfully

--- DOUBLY LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter element: 15
15 inserted at end
```


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
--- DOUBLY LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter element: 16
Enter index: 3
16 inserted at index 3
--- DOUBLY LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 15
15 deleted from the list
--- DOUBLY LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
--- DOUBLY LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
12 deleted from beginning
--- DOUBLY LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
16 deleted from end
--- DOUBLY LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit




```
--- DOUBLY LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program terminated
>>>
```

EXPLANATION:

1. self.prev = None — this snippet gives every node an extra link to the node before it, so the list can be traversed in both directions.
2. self.head.prev = new_node — this snippet makes the old first node point back to the new node when inserting at the beginning.
3. new_node.prev = temp — this snippet makes the newly added last node remember the node before it.
4. if temp.next is not None: temp.next.prev = new_node — this snippet makes the node after the insertion point point back to the new node. The check skips this step when inserting at the end.
5. while temp is not None and temp.data != value: — this snippet stops on the node to be deleted itself, not one node before it, because the previous node can be reached through temp.prev.
6. temp.prev.next = temp.next — this snippet makes the previous node skip over the deleted node in the forward direction.
7. temp.next.prev = temp.prev — this snippet makes the next node skip over the deleted node in the backward direction, so both links are repaired.
8. else: self.head = temp.next — this snippet moves head forward when the deleted node has no previous node, because that means it was the first node.
9. self.head.prev = None — this snippet clears the backward link of the new first node after the old first node is deleted.
10. temp.prev.next = None — this snippet deletes the last node by going straight to it and cutting the link from its previous node, without needing temp.next.next.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Forgot self.head.prev = new_node in insert at beginning	Logical	Deleting the second element also removes the first element (e.g., the list becomes None <-> 20 <-> None)	Add self.head.prev = new_node before changing self.head
Left out the check if temp.next is not None before temp.next.prev = new_node	Runtime — AttributeError	AttributeError: 'NoneType' object has no attribute 'prev'	Update temp.next.prev only when temp.next is not None
Forgot self.head.prev = None after deleting the first node	Logical	"20 deleted from the list" is printed but 20 still appears in the display	Set self.head.prev = None when the head moves forward
Did not write self.prev = None in the Node class	Runtime — AttributeError	AttributeError: 'Node' object has no attribute 'prev'	Add self.prev = None in the Node constructor

C.AIM:

To write a menu-driven Python program that implements a circular linked list and performs these operations: create, insert at beginning, insert at end, insert at index, delete by value, delete first node, delete last node, count nodes, display (traverse) and exit.

INPUT CODE:

3.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-5/3.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class CircularLinkedList:
    def __init__(self):
        self.head = None
    def create(self):
        n = int(input("Enter number of nodes: "))
        if n <= 0:
            print("Number of nodes must be greater than 0")
            return
        for i in range(n):
            data = int(input("Enter element: "))
            new_node = Node(data)
            if self.head is None:
                self.head = new_node
                new_node.next = self.head
            else:
                temp = self.head
                while temp.next != self.head:
                    temp = temp.next
                temp.next = new_node
                new_node.next = self.head
        print("Circular linked list created successfully")
    def insert_beginning(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            new_node.next = new_node
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            new_node.next = self.head
            temp.next = new_node
            self.head = new_node
        print(data, "inserted at beginning")
    def insert_end(self, data):
```

3.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-5/3.py (3.14.0)

File Edit Format Run Options Window Help

```
        if self.head is None:
            self.head = new_node
            new_node.next = new_node
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = new_node
            new_node.next = self.head
        print(data, "inserted at end")
    def insert_index(self, data, index):
        if index < 0:
            print("Invalid index")
            return
        if index == 0:
            self.insert_beginning(data)
            return
        if self.head is None:
            print("Index out of range")
            return
        temp = self.head
        for i in range(index - 1):
            temp = temp.next
            if temp == self.head:
                print("Index out of range")
                return
        new_node = Node(data)
        new_node.next = temp.next
        temp.next = new_node
        print(data, "inserted at index", index)
    def delete_value(self, value):
        if self.head is None:
            print("Linked list is empty")
            return
        if self.head.data == value:
            if self.head.next == self.head:
                self.head = None
            else:
                temp = self.head
```

3.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-5/3.py (3.14.0)

File Edit Format Run Options Window Help

```
                    temp = temp.next
                temp.next = self.head.next
                self.head = self.head.next
            print(value, "deleted from the list")
            return
        prev = self.head
        temp = self.head.next
        while temp != self.head:
            if temp.data == value:
                prev.next = temp.next
                print(value, "deleted from the list")
                return
            prev = temp
            temp = temp.next
        print(value, "not found in the list")
    def delete_first(self):
        if self.head is None:
            print("Linked list is empty")
            return
        value = self.head.data
        if self.head.next == self.head:
            self.head = None
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = self.head.next
            self.head = self.head.next
        print(value, "deleted from beginning")
    def delete_last(self):
        if self.head is None:
            print("Linked list is empty")
            return
        if self.head.next == self.head:
            value = self.head.data
            self.head = None
            print(value, "deleted from end")
            return
        temp = self.head
        while temp.next.next != self.head:
```

3.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-5/3.py (3.14.0)

File Edit Format Run Options Window Help

```
            temp = temp.next
        value = temp.next.data
        temp.next = self.head
        print(value, "deleted from end")
    def count(self):
        count = 0
        if self.head is not None:
            temp = self.head
            while True:
                count += 1
                temp = temp.next
                if temp == self.head:
                    break
        print("Number of nodes:", count)
    def display(self):
        if self.head is None:
            print("Linked list is empty")
            return
        temp = self.head
        print("Circular linked list:")
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break
        print(self.head.data)
cll = CircularLinkedList()
while True:
    print("\n--- CIRCULAR LINKED LIST MENU ---")
    print("1. Create")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")
    choice = int(input("Enter your choice: "))
```

3.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-5/3.py (3.14.0)

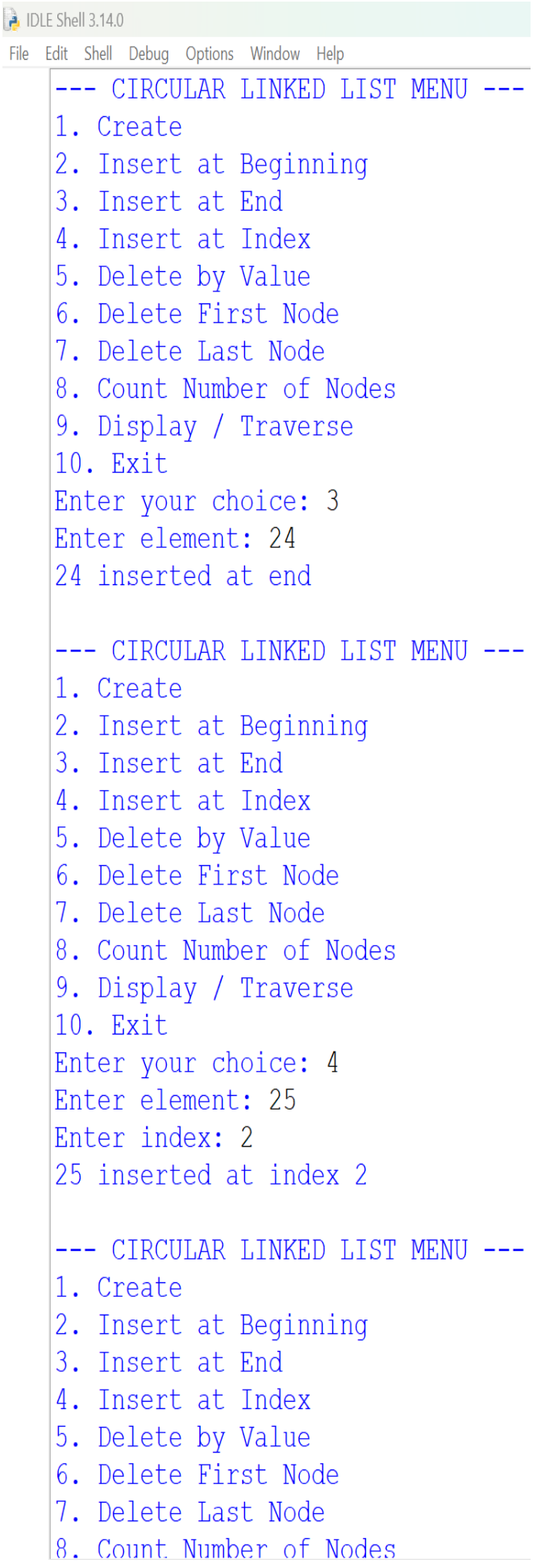
File Edit Format Run Options Window Help

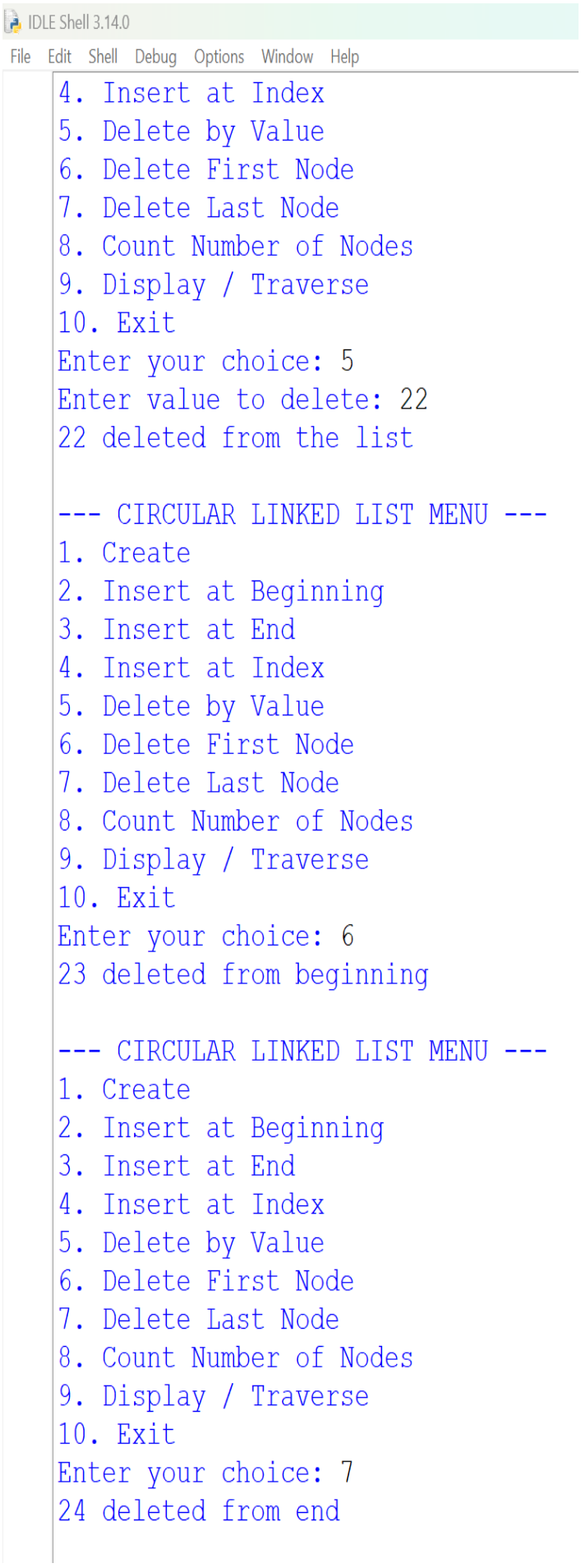
```
    print("1. Create")
    print("2. Insert at Beginning")
    print("3. Insert at End")
    print("4. Insert at Index")
    print("5. Delete by Value")
    print("6. Delete First Node")
    print("7. Delete Last Node")
    print("8. Count Number of Nodes")
    print("9. Display / Traverse")
    print("10. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        cll.create()
    elif choice == 2:
        data = int(input("Enter element: "))
        cll.insert_beginning(data)
    elif choice == 3:
        data = int(input("Enter element: "))
        cll.insert_end(data)
    elif choice == 4:
        data = int(input("Enter element: "))
        index = int(input("Enter index: "))
        cll.insert_index(data, index)
    elif choice == 5:
        value = int(input("Enter value to delete: "))
        cll.delete_value(value)
    elif choice == 6:
        cll.delete_first()
    elif choice == 7:
        cll.delete_last()
    elif choice == 8:
        cll.count()
    elif choice == 9:
        cll.display()
    elif choice == 10:
        print("Program terminated")
        break
    else:
        print("Invalid choice")

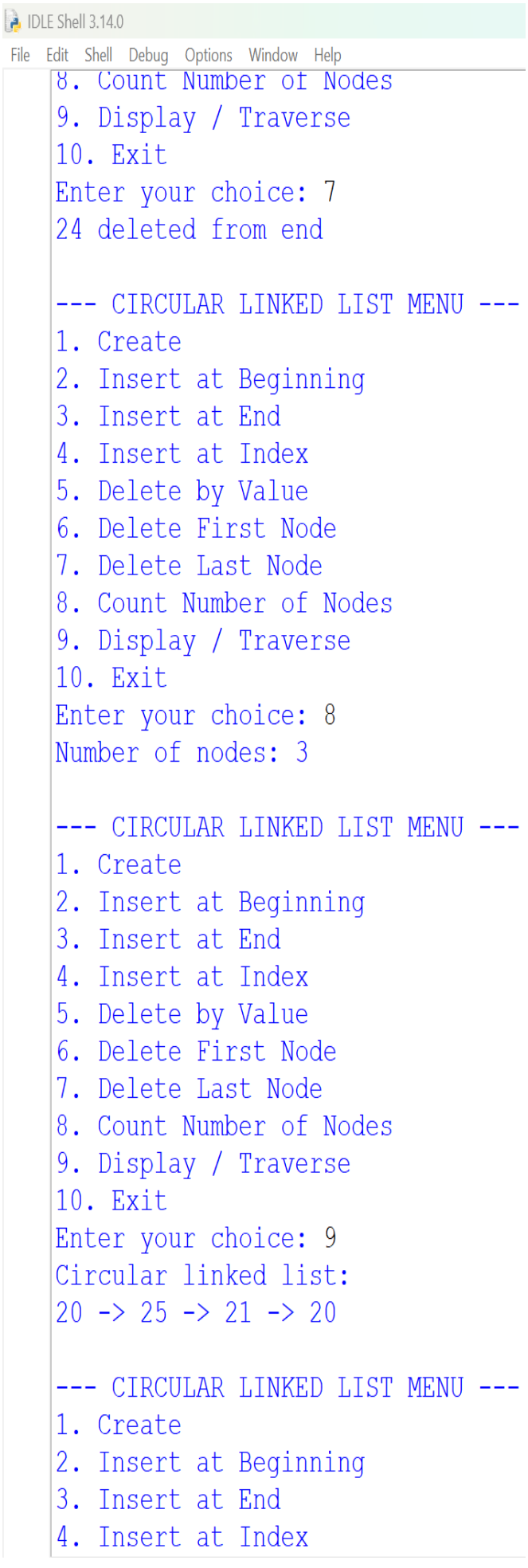
```

OUTPUT:


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-5/3.py ==========
--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 1
Enter number of nodes: 3
Enter element: 20
Enter element: 21
Enter element: 22
Circular linked list created successfully
--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 2
Enter element: 23
23 inserted at beginning
--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 3
Enter element: 24
24 inserted at end
--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 4
Enter element: 25
Enter index: 2
25 inserted at index 2
--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 5
Enter value to delete: 22
22 deleted from the list
--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 6
23 deleted from beginning
--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
24 deleted from end


IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 7
24 deleted from end
--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 8
Number of nodes: 3
--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 9
Circular linked list:
20 -> 25 -> 21 -> 20
--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index

```
10. Exit
Enter your choice: 9
Circular linked list:
20 -> 25 -> 21 -> 20

--- CIRCULAR LINKED LIST MENU ---
1. Create
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete First Node
7. Delete Last Node
8. Count Number of Nodes
9. Display / Traverse
10. Exit
Enter your choice: 10
Program terminated
>>>
```

EXPLANATION:

1. new_node.next = self.head — this snippet makes the newly added last node point back to the head instead of None, which keeps the list circular.
2. new_node.next = new_node — this snippet makes the only node in an empty list point to itself, forming a circle of one node.
3. temp.next = new_node / self.head = new_node — this snippet updates the last node to point to the new first node when inserting at the beginning, which a singly linked list does not need.
4. if temp == self.head: print("Index out of range") — this snippet detects that the walk has gone round the full circle and come back to head, which means the index is larger than the list.
5. if self.head.next == self.head: — this snippet checks whether the list has only one node, which is true when that node points to itself.
6. temp.next = self.head.next / self.head = self.head.next — this snippet deletes the head by linking the last node to the second node and making the second node the new head.
7. prev = self.head / temp = self.head.next / while temp != self.head: — this snippet searches using two pointers and stops after exactly one full round of the circle.
8. temp.next = self.head (in delete_last) — this snippet makes the second-last node the new last node by linking it back to the head.

9. while True: … if temp == self.head: break — this snippet works like a do-while loop. It visits the head first and visits every node exactly once before stopping.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Used the singly list condition while temp.next is not None in insert at end	Logical	The program hangs with no output, and pressing Ctrl+C shows KeyboardInterrupt	Use while temp.next != self.head: in a circular list
Wrote the display loop as while temp != self.head:	Logical	Circular linked list: is followed by only the first value and no other elements	Use while True: with if temp == self.head: break at the end of the loop
Counted with while temp.next != self.head: starting from count = 0	Logical	Number of nodes is one less than the actual count (e.g., 2 instead of 3)	Use the while True: loop so the last node is also counted
Left out the if temp == self.head: check in insert at index	Logical	The element is inserted at a wrong position instead of showing "Index out of range"	Check if temp == self.head: inside the loop and stop

WEEK – 6

A.AIM:

Write a program to implement stack operations such as push, pop peek, display using an array.

INPUT:

1.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-6/1.py (3.14.0)

File Edit Format Run Options Window Help

```
class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = [0] * size
        self.top = -1
    def isEmpty(self):
        return self.top == -1
    def isFull(self):
        return self.top == self.size - 1
    def push(self, item):
        if self.isFull():
            print("Stack Overflow")
        else:
            self.top = self.top + 1
            self.stack[self.top] = item
            print(item, "pushed into stack")
    def pop(self):
        if self.isEmpty():
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.top = self.top - 1
            print(item, "popped from stack")
    def peek(self):
        if self.isEmpty():
            print("Stack is empty")
        else:
            print("Top element is", self.stack[self.top])
    def display(self):
        if self.isEmpty():
            print("Stack is empty")
        else:
            print("Stack elements are:")
            for i in range(self.top, -1, -1):
                print(self.stack[i])
size = int(input("Enter size of stack: "))
s = Stack(size)
```

```
s = Stack(size)
while True:
    print("1.Push\n2.Pop\n3.Peek\n4.Display\n5.Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT :

```
Enter "help" below or click
>>>
========== RESTART: C:/Us
Enter size of stack: 6
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to push: 12
12 pushed into stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to push: 7
7 pushed into stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to push: 9
9 pushed into stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 2
9 popped from stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 3
Top element is 7
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 4
Stack elements are:
7
12
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 5
```

EXPLANATION:

1. self.stack = [0] * size — this snippet creates a fixed-size array that holds the stack elements.
2. self.top = -1 — this snippet sets top to -1, which means the stack is empty at the start.
3. return self.top == self.size - 1 — this snippet checks whether top has reached the last index, which means the stack is full.
4. if self.isFull(): print("Stack Overflow") — this snippet stops a push when there is no space left in the array.
5. self.top = self.top + 1 / self.stack[self.top] = item — this snippet moves top up by one and stores the new element in that position.
6. item = self.stack[self.top] / self.top = self.top - 1 — this snippet reads the top element and moves top down by one, which removes it from the stack.
7. print("Top element is", self.stack[self.top]) — this snippet shows the top element without removing it (peek).
8. for i in range(self.top, -1, -1): — this snippet prints the elements from top to bottom, following the Last In First Out (LIFO) order.
9. s = Stack(size) — this snippet creates a stack object with the size the user entered.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Pushed without the if self.isFull() check	Runtime — IndexError	IndexError: list assignment index out of range	Check if self.isFull(): print("Stack Overflow") before pushing
Popped without the if self.isEmpty() check	Logical	0 popped from stack (a wrong value is popped even though the stack is empty)	Check if self.isEmpty(): print("Stack Underflow") before popping
Stored the item before increasing top (self.stack[self.top] = item written before self.top = self.top + 1)	Logical	Top element is 0 (the wrong top element is shown after a push)	Increase self.top first, then store self.stack[self.top] = item

B.AIM:To write a Python program that implements a stack using a linked list and performs the push, pop, peek and display operations using a menu.

INPUT CODE

2.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-6/2.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Stack:
    def __init__(self):
        self.top = None
    def isEmpty(self):
        return self.top is None
    def push(self, item):
        newNode = Node(item)
        newNode.next = self.top
        self.top = newNode
        print(item, "pushed into stack")
    def pop(self):
        if self.isEmpty():
            print("Stack Underflow")
        else:
            temp = self.top
            self.top = self.top.next
            print(temp.data, "popped from stack")
    def peek(self):
        if self.isEmpty():
            print("Stack is empty")
        else:
            print("Top element is", self.top.data)
    def display(self):
        if self.isEmpty():
            print("Stack is empty")
        else:
            print("Stack elements are:")
            temp = self.top
            while temp is not None:
                print(temp.data)
                temp = temp.next
:s = Stack()
s = Stack()
while True:
    print("1.Push\n2.Pop\n3.Peek\n4.Display\n5.Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
>>>
============ RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-6/2.p
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to push: 34
34 pushed into stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to push: 12
12 pushed into stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 2
12 popped from stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 3
Top element is 34
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 4
Enter your choice: 4
Stack elements are:
34
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 5
```

EXPLANATION:

1. class Stack: / def __init__(self): — this snippet creates a stack with no fixed size. Memory is used only when an element is pushed.

2. self.top = None — this snippet makes top a pointer to the topmost node. None means the stack has no nodes.

3. return self.top is None — this snippet checks for an empty stack by looking at the top pointer instead of an index.

4. newNode = Node(item) — this snippet creates a new node for every push, so the stack never overflows the way a fixed array can.

5. newNode.next = self.top — this snippet links the new node to the current top, so the older elements stay below it.

6. self.top = newNode — this snippet makes the new node the top of the stack, so every push happens at the front.

7. temp = self.top — this snippet saves the top node before removing it, so its value can still be printed after the pop.

8. self.top = self.top.next — this snippet moves top down to the next node. Nothing points to the old top anymore, so Python frees its memory automatically.

9. print("Top element is", self.top.data) — this snippet reads the data stored in the top node without changing the stack.

10. while temp is not None: — this snippet uses a separate pointer called temp to go from the top node to the bottom node, so top itself is never changed while displaying.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Called peek on an empty stack without the if self.isEmpty() check	Runtime — AttributeError	AttributeError: 'NoneType' object has no attribute 'data	Check if self.isEmpty(): print("Stack is empty") before reading self.top.data
Created the object as s = Stack without brackets	Runtime — TypeError	TypeError: Stack.push() missing 1 required positional argument: 'item'	Create the object as s = Stack()
Left out newNode.next = self.top in push	Logical	Only the last pushed element is shown and all earlier elements are lost	Link newNode.next = self.top before setting self.top = newNode

WEEK-7

A. AIM:

To write a menu-driven Python program that implements a queue using an array and performs the enqueue, dequeue, peek and display operations.

INPUT CODE:

1.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-7/1.py (3.14.0)

File Edit Format Run Options Window Help

```
class Queue:
    def __init__(self, size):
        self.size = size
        self.queue = [0] * size
        self.front = -1
        self.rear = -1
    def isEmpty(self):
        return self.front == -1
    def isFull(self):
        return self.rear == self.size - 1
    def enqueue(self, item):
        if self.isFull():
            print("Queue Overflow")
        else:
            if self.front == -1:
                self.front = 0
            self.rear = self.rear + 1
            self.queue[self.rear] = item
            print(item, "inserted into queue")
    def dequeue(self):
        if self.isEmpty():
            print("Queue Underflow")
        else:
            item = self.queue[self.front]
            self.front = self.front + 1
            if self.front > self.rear:
                self.front = self.rear = -1
            print(item, "deleted from queue")
    def peek(self):
        if self.isEmpty():
            print("Queue is empty")
        else:
            print("Front element is", self.queue[self.front])
    def display(self):
        if self.isEmpty():
            print("Queue is empty")
        else:
            print("Queue elements are:")
            for i in range(self.front, self.rear + 1):
```

```
            for i in range(self.front, self.rear + 1):
                print(self.queue[i], end=" ")
            print()
size = int(input("Enter size of queue: "))
q = Queue(size)
while True:
    print("1.Enqueue\n2.Dequeue\n3.Peek\n4.Display\n5.Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter element to insert: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
====================================== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-7/1.py
Enter size of queue: 5
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to insert: 12
12 inserted into queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to insert: 23
23 inserted into queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 2
12 deleted from queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 3
Front element is 23
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 4
```

```
5.Exit
Enter your choice: 4
Queue elements are:
23
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 5
>>>
```

EXPLANATION:

1. self.front = -1 / self.rear = -1 — this snippet sets up two indexes: front marks where elements are deleted and rear marks where they are inserted. Both being -1 means the queue is empty.

2. return self.front == -1 — this snippet checks whether the queue is empty by looking at the front index.

3. if self.front == -1: self.front = 0 — this snippet moves front to index 0 when the first element is inserted.

4. self.rear = self.rear + 1 / self.queue[self.rear] = item — this snippet adds the new element at the rear end, so elements are kept in the order they arrive.

5. item = self.queue[self.front] / self.front = self.front + 1 — this snippet removes the element at the front, so the element inserted first is removed first (First In First Out, FIFO).

6. if self.front > self.rear: self.front = self.rear = -1 — this snippet resets both indexes when the last element is removed, so the queue is empty again.

7. print("Front element is", self.queue[self.front]) — this snippet shows the element that will be removed next, without deleting it.

8. for i in range(self.front, self.rear + 1): — this snippet prints only the valid part of the array, from front to rear, and skips the positions that were already dequeued.

9. while True: — this snippet shows the menu again and again until the user chooses Exit.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Forgot to set self.front = 0 on the first enqueue	Logical	Front element is 0 (a wrong front element is shown and an extra 0 appears in the display)	Add if self.front == -1: self.front = 0 before inserting
Forgot to reset self.front = self.rear = -1 after the last dequeue	Logical	Queue elements are: (a blank line is printed instead of "Queue is empty")	Add if self.front > self.rear: self.front = self.rear = -1
Wrote self.rear = self.rear - 1 instead of self.front = self.front + 1 in dequeue	Logical	The last inserted element is deleted instead of the first one (it behaves like a stack)	Delete from the front using self.front = self.front + 1
Tried to insert after dequeuing some elements in a full linear queue	Logical	Queue Overflow (shown even though there are free positions at the front)	Use a circular queue with self.rear = (self.rear + 1) % self.size

B.AIM:

To write a menu-driven Python program that implements a queue using a linked list and performs the enqueue, dequeue, peek and display operations.

INPUT CODE:

2.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-7/2.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
    def isEmpty(self):
        return self.front is None
    def enqueue(self, item):
        newNode = Node(item)
        if self.rear is None:
            self.front = self.rear = newNode
        else:
            self.rear.next = newNode
            self.rear = newNode
        print(item, "inserted into queue")
    def dequeue(self):
        if self.isEmpty():
            print("Queue Underflow")
        else:
            temp = self.front
            self.front = self.front.next
            if self.front is None:
                self.rear = None
            print(temp.data, "deleted from queue")
    def peek(self):
        if self.isEmpty():
            print("Queue is empty")
        else:
            print("Front element is", self.front.data)
    def display(self):
        if self.isEmpty():
            print("Queue is empty")
        else:
            print("Queue elements are:")
            temp = self.front
```

```
            temp = self.front
            while temp:
                print(temp.data, end=" ")
                temp = temp.next
            print()
q = Queue()
while True:
    print("1.Enqueue\n2.Dequeue\n3.Peek\n4.Display\n5.Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter element to insert: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-7/2.py
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to insert: 12
12 inserted into queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 34
Invalid choice
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to insert: 34
34 inserted into queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 2
12 deleted from queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 3
Front element is 34
```

```
Enter element to insert: 34
34 inserted into queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 2
12 deleted from queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 3
Front element is 34
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 4
Queue elements are:
34
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 5
```

EXPLANATION:

1. self.front = None / self.rear = None — this snippet keeps two pointers: front points to the first node and rear points to the last node.

2. if self.rear is None: self.front = self.rear = newNode — this snippet makes the first node both the front and the rear, because it is the only node in the queue.

3. self.rear.next = newNode — this snippet links the current last node to the new node, which adds the new node at the end.

4. self.rear = newNode — this snippet moves rear to the new node, so the next insertion does not have to go through the whole list.

5. self.front = self.front.next — this snippet deletes from the front, the end opposite to where insertion happens, which keeps the FIFO order.

6. if self.front is None: self.rear = None — this snippet also clears rear when the last node is removed, so rear does not keep pointing to a deleted node.

7. print("Front element is", self.front.data) — this snippet shows the data of the first node, the one that has waited the longest in the queue.

8. choice = int(input("Enter your choice: ")) — this snippet reads the user's menu option as a number so it can be matched with the if–elif conditions.

9. elif choice == 5: break — this snippet stops the menu loop and ends the program when the user chooses Exit.

10. else: print("Invalid choice") — this snippet handles any number that is not one of the menu options.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Forgot self.rear = None when the last node is dequeued	Logical	Queue is empty (shown even after inserting a new element)	Add if self.front is None: self.rear = None in dequeue
Forgot self.rear = newNode after self.rear.next = newNode		Only the first and last inserted elements are displayed and the middle elements are lost	Update self.rear = newNode after linking the new node
Defined the method as def dequeue(): without self	Runtime — TypeError	TypeError: Queue.dequeue() takes 0 positional arguments but 1 was given	Write def dequeue(self):
Did not indent the line under an if statement	Syntax — IndentationError	IndentationError: expected an indented block after 'if' statement on line X	Indent the lines inside the if block by 4 spaces

C.AIM:

To write a menu-driven Python program that implements a circular queue using an array and performs the enqueue, dequeue, peek and display operations.

INPUT CODE:

2.1.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-7/2.1.py (3.14.0)

File Edit Format Run Options Window Help

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = self.rear = -1
    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into queue")
        else:
            self.rear = (self.rear + 1) % self.size
            self.queue[self.rear] = item
            print(item, "inserted into queue")
    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")
        elif self.front == self.rear:
            item = self.queue[self.front]
            self.front = self.rear = -1
            print(item, "deleted from queue")
        else:
            item = self.queue[self.front]
            self.front = (self.front + 1) % self.size
            print(item, "deleted from queue")
    def peek(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Front element is", self.queue[self.front])
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("Queue elements are:")
            if self.rear >= self.front:
```

```
            if self.rear >= self.front:
                for i in range(self.front, self.rear + 1):
                    print(self.queue[i], end=" ")
            else:
                for i in range(self.front, self.size):
                    print(self.queue[i], end=" ")
                for i in range(0, self.rear + 1):
                    print(self.queue[i], end=" ")
            print()
size = int(input("Enter size of queue: "))
q = CircularQueue(size)
while True:
    print("1.Enqueue\n2.Dequeue\n3.Peek\n4.Display\n5.Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter element to insert: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
>>>
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-7/2.1.p
Enter size of queue: 4
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to insert: 12
12 inserted into queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to insert: 14
14 inserted into queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 2
12 deleted from queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 3
Front element is 14
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
```

```
5.Exit
Enter your choice: 3
Front element is 14
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 4
Queue elements are:
14
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 5
>>>
```

EXPLANATION:

1. if (self.rear + 1) % self.size == self.front: — this snippet checks for a full queue: if the next position after rear is front, there is no free space left.

2. self.rear = (self.rear + 1) % self.size — this snippet moves rear forward. After the last index it wraps back to index 0, so the free spaces at the start can be used again.

3. self.front = (self.front + 1) % self.size — this snippet moves front forward in the same circular way, going back to index 0 after the last index.

4. elif self.front == self.rear: — this snippet finds the case where only one element is left. After removing it, both indexes are set back to -1.

5. if self.rear >= self.front: — this snippet checks whether the elements sit in one continuous block (no wrap-around yet), so they can be printed in a single pass.

6. for i in range(self.front, self.size): — this snippet prints the elements from front to the end of the array when the queue has wrapped around.

7. for i in range(0, self.rear + 1): — this snippet prints the remaining elements from index 0 up to rear, which completes the circular display.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Left out the elif self.front == self.rear case in dequeue	Logical	Queue Overflow (shown when inserting into a queue that is actually empty)	Add elif self.front == self.rear: self.front = self.rear = -1
Used / instead of % in (self.rear + 1) % self.size	Runtime — TypeError	TypeError: list indices must be integers or slices, not float	Use the modulus operator % to wrap the index
Used % (self.size - 1) instead of % self.size	Logical	Queue Overflow	Use % self.size so every position is used

D.AIM:

To write a menu-driven Python program that implements a circular queue using a linked list and performs the enqueue, dequeue, peek and display operations.

INPUT:

2.2.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-7/2.2.py (3.14.0)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class CircularQueue:
    def __init__(self):
        self.front = self.rear = None
    def enqueue(self, item):
        newNode = Node(item)
        if self.front is None:
            self.front = newNode
        else:
            self.rear.next = newNode
        self.rear = newNode
        self.rear.next = self.front
        print(item, "inserted into queue")
    def dequeue(self):
        if self.front is None:
            print("Queue Underflow")
        elif self.front == self.rear:
            item = self.front.data
            self.front = self.rear = None
            print(item, "deleted from queue")
        else:
            item = self.front.data
            self.front = self.front.next
            self.rear.next = self.front
            print(item, "deleted from queue")
    def peek(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Front element is", self.front.data)
    def display(self):
        if self.front is None:
            print("Queue is empty")
        else:
            print("Queue elements are:")
            temp = self.front
```

```
            temp = self.front
            while temp.next != self.front:
                print(temp.data, end=" ")
                temp = temp.next
            print(temp.data)
q = CircularQueue()
while True:
    print("1.Enqueue\n2.Dequeue\n3.Peek\n4.Display\n5.Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter element to insert: "))
        q.enqueue(item)
    elif choice == 2:
        q.dequeue()
    elif choice == 3:
        q.peek()
    elif choice == 4:
        q.display()
    elif choice == 5:
        break
    else:
        print("Invalid choice")
```

OUTPUT:

```
>>>
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-7/2.2.py ==========
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to insert: 12
12 inserted into queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 2
12 deleted from queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 3
Queue is empty
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 4
Queue is empty
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 5
```

EXPLANATION:

1. self.rear.next = self.front (in enqueue) — this snippet links the last node back to the first node after every insertion, which makes the list circular.

2. if self.front is None: self.front = newNode / else: self.rear.next = newNode — this snippet decides where the new node goes. If the queue is empty it becomes the front, otherwise it is attached after the current rear.

3. elif self.front == self.rear: self.front = self.rear = None — this snippet handles the single-node case. That node points to itself, so both pointers must be cleared to empty the queue.

4. self.rear.next = self.front (in dequeue) — this snippet links rear to the new front after a deletion, so the circle stays unbroken without the removed node.

5. item = self.front.data — this snippet saves the value of the front node before the node is removed from the circle.

6. while temp.next != self.front: — this snippet stops the display when it reaches the node that points back to front. A circular list has no None at the end, so this is the stopping condition.

7. print(temp.data) — this snippet prints the last node after the loop, because the loop condition stops just before printing it.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Forgot self.rear.next = self.front in dequeue	Logical	The deleted element still appears at the end of the display (e.g., 20 30 10)	Relink with self.rear.next = self.front after moving front
Left out the elif self.front == self.rear case in dequeue	Logical	The same element is shown as deleted again and again and the queue never becomes empty	Add elif self.front == self.rear: self.front = self.rear = None
Wrote the display loop as while temp: instead of while temp.next != self.front:	Logical	The elements keep printing in a circle without stopping	Use while temp.next != self.front:

WEEK-8

A.AIM:

Write a program to implement a binary tree with its operations using an array.

INPUT CODE:

8.1.py - C:/Users/Lab/Desktop/CSE25113/8.1.py (3.11.4)

File Edit Format Run Options Window Help

```
class BinaryTree:
    def __init__(self):
        self.tree = []

    def insert(self, value):
        self.tree.append(value)

    def search(self, value):
        if value in self.tree:
            print("Found at index:", self.tree.index(value))
        else:
            print("Value not found")

    def delete(self, value):
        if value in self.tree:
            self.tree.remove(value)
            print("Deleted:", value)
        else:
            print("Value not found")

    def preorder(self, i=0):
        if i < len(self.tree):
            print(self.tree[i], end=" ")
            self.preorder(2 * i + 1)
            self.preorder(2 * i + 2)

    def display(self):
        print(self.tree)
bt = BinaryTree()
bt.insert(10)
bt.insert(20)
bt.insert(30)
bt.insert(40)
bt.insert(50)

print("Array:", end=" ")
bt.display()

print("Preorder:", end=" ")
bt.preorder()

print()
bt.search(30)
bt.delete(20)
bt.display()
```

OUTPUT PAGE:

```
= RESTART: C:/Users/Lab/Desktop/CSE25113/8.1.py
Array: [10, 20, 30, 40, 50]
Preorder: 10 20 40 50 30
Found at index: 2
Deleted: 20
[10, 30, 40, 50]
>>>
```

EXPLANATION:

1. self.tree = [] — this snippet creates an empty Python list to store the binary tree elements.
2. def insert(self, value): — this snippet defines the insertion operation for adding values to the tree.
3. self.tree.append(value) — this snippet adds a new value to the end of the array.
4. if value in self.tree: — this snippet checks whether the requested value exists in the array.
5. self.tree.remove(value) — this snippet deletes the first occurrence of the specified value.
6. def preorder(self, i=0): — this snippet defines preorder traversal, starting at index 0.
7. self.preorder(2 * i + 1) and self.preorder(2 * i + 2) — these snippets recursively visit the left and right child positions.
8. bt.display() — this snippet displays the elements stored in the array.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Initialized the array with a fixed size using [None] * size	Logical	The array displays many None values in unused positions	Use self.tree = [] and self.tree.append(value) to avoid displaying unused positions.
Used an incorrect index for the left child	Logical	The preorder traversal visits nodes in the wrong order	Use 2 * i + 1 to calculate the left-child index.
Used remove(value) to delete a node without considering tree indices	Logical	Parent-child relationships change after deletion	Set the selected position to None and handle the affected subtree appropriately.

B.AIM:

Write a program to implement a binary tree with its operations using a linked list.

INPUT CODE:

8.1.2.py - C:/Users/Lab/Desktop/CSE25113/8.1.2.py (3.11.4)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
class BinaryTreeLinkedList:
    def __init__(self):
        self.root = None

    def insert(self, value):
        new_node = Node(value)

        if self.root is None:
            self.root = new_node
            print("Inserted:", value)
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

            if current.left is None:
                current.left = new_node
                print("Inserted:", value)
                return
            else:
                queue.append(current.left)

            if current.right is None:
                current.right = new_node
                print("Inserted:", value)
                return
            else:
                queue.append(current.right)

    def search(self, value):
        if self.root is None:
            print("Value not found")
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)

```

8.1.2.py - C:/Users/Lab/Desktop/CSE25113/8.1.2.py (3.11.4)

File Edit Format Run Options Window Help

```
            if current.data == value:
                print("Value found:", value)
                return

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)

        print("Value not found")

    def delete(self, value):
        if self.root is None:
            print("Tree is empty")
            return

        queue = [self.root]
        target = None
        deepest = None
        parent_of_deepest = None

        while queue:
            current = queue.pop(0)

            if current.data == value and target is None:
                target = current

            if current.left:
                parent_of_deepest = current
                deepest = current.left
                queue.append(current.left)

            if current.right:
                parent_of_deepest = current
                deepest = current.right
                queue.append(current.right)

        if target is None:
            print("Value not found")
            return

        if deepest is None:
            self.root = None
            print("Deleted:", value)
```

8.1.2.py - C:/Users/Lab/Desktop/CSE25113/8.1.2.py (3.11.4)

File Edit Format Run Options Window Help

```
                print("Deleted:", value)
                return

            target.data = deepest.data

            if parent_of_deepest.left is deepest:
                parent_of_deepest.left = None
            else:
                parent_of_deepest.right = None

            print("Deleted:", value)

    def preorder(self, node):
        if node:
            print(node.data, end=" ")
            self.preorder(node.left)
            self.preorder(node.right)

    def inorder(self, node):
        if node:
            self.inorder(node.left)
            print(node.data, end=" ")
            self.inorder(node.right)

    def postorder(self, node):
        if node:
            self.postorder(node.left)
            self.postorder(node.right)
            print(node.data, end=" ")

    def display(self):
        if self.root is None:
            print("Tree is empty")
            return

        queue = [self.root]

        while queue:
            current = queue.pop(0)
            print(current.data, end=" ")

            if current.left:
                queue.append(current.left)

            if current.right:
                queue.append(current.right)
```

```
bt = BinaryTreeLinkedList()
bt.insert(10)
bt.insert(20)
bt.insert(30)
bt.insert(40)
bt.insert(50)

print("Level-order traversal:")
bt.display()

print("\nPreorder traversal:")
bt.preorder(bt.root)

print("\nInorder traversal:")
bt.inorder(bt.root)

print("\nPostorder traversal:")
bt.postorder(bt.root)

print()
bt.search(30)
bt.delete(20)

print("Tree after deletion:")
bt.display()

```

OUTPUT:

```
>>>
========== RESTART: C:/Users/Lab/Desktop/CSE25113/8.1.2.py
Inserted: 10
Inserted: 20
Inserted: 30
Inserted: 40
Inserted: 50
Level-order traversal:
10 20 30 40 50
Preorder traversal:
10 20 40 50 30
Inorder traversal:
40 20 50 10 30
Postorder traversal:
40 50 20 30 10
Value found: 30
Deleted: 20
Tree after deletion:
10 50 30 40
>>>
```

EXPLANATION:

1. class Node: — this snippet defines a node that stores data and references to its left and right children.
2. self.data = data — this snippet stores the value inside the node.
3. self.left = None and self.right = None — these snippets initialize the left and right child references as empty.
4. self.root = None — this snippet initializes the tree as empty, with no root node.
5. new_node = Node(value) — this snippet creates a new node containing the value to be inserted.
6. current.left = new_node or current.right = new_node — these snippets connect the new node to the first available child position.
7. if current.data == value: — this snippet checks whether the current node contains the value being searched for.
8. target.data = deepest.data — this snippet replaces the target node's value with the deepest node's value during deletion, after which the deepest node is removed.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Forgot to set self.root = new_node when the tree is empty	Logical	The first node is not stored as the root, and the tree remains empty	Add self.root = new_node inside the empty-tree condition.
Forgot the base condition if node: in traversal	Runtime — AttributeError	AttributeError: 'NoneType' object has no attribute 'data'	Add if node: before accessing the node's data or children.
Forgot to check whether the target value exists before deletion	Runtime — AttributeError	An error occurs when attempting to access target.data for a missing value	Check if target is None: and return before deleting.
Forgot to initialize self.left and self.right in the Node class	Runtime — AttributeError	AttributeError: 'Node' object has no attribute 'left'	Initialize both child references with self.left = None and self.right = None.

WEEK-9

AIM:

Write a python program to implement a binary search tree with its operations and traversals [inorder, preorder, postorder].

INPUT CODE:

1.py - C:/Users/Lab/Desktop/CSE25113/WEEK-9/1.py (3.11.4)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
class BST:
    def __init__(self):
        self.root = None

    def insert(self, root, value):
        if root is None:
            return Node(value)

        if value < root.data:
            root.left = self.insert(root.left, value)
        elif value > root.data:
            root.right = self.insert(root.right, value)

        return root

    def search(self, root, value):
        if root is None or root.data == value:
            return root

        if value < root.data:
            return self.search(root.left, value)

        return self.search(root.right, value)

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")
```

1.py - C:/Users/Lab/Desktop/CSE25113/WEEK-9/1.py (3.11.4)

File Edit Format Run Options Window Help

```
            print(root.data, end=" ")

    def min_value(self, root):
        while root.left:
            root = root.left
        return root

    def delete(self, root, value):
        if root is None:
            return root

        if value < root.data:
            root.left = self.delete(root.left, value)
        elif value > root.data:
            root.right = self.delete(root.right, value)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left

            temp = self.min_value(root.right)
            root.data = temp.data
            root.right = self.delete(root.right, temp.data)

        return root
bst = BST()

for value in [50, 30, 70, 20, 40, 60, 80]:
    bst.root = bst.insert(bst.root, value)

print("Inorder traversal:")
bst.inorder(bst.root)

print("\nPreorder traversal:")
bst.preorder(bst.root)

print("\nPostorder traversal:")
bst.postorder(bst.root)

print("\n")

if bst.search(bst.root, 40):
    print("Value found")
else:
    print("Value not found")
```

```
bst.root = bst.delete(bst.root, 30)

print("Inorder after deletion:")
bst.inorder(bst.root)
```

OUTPUT:

```
= RESTART: C:/Users/Lab/Desktop/CSE25113/WEEK-9/1.py
Inorder traversal:
20 30 40 50 60 70 80
Preorder traversal:
50 30 20 40 70 60 80
Postorder traversal:
20 40 30 60 80 70 50

Value found
Inorder after deletion:
20 40 50 60 70 80
>>>
```

EXPLANATION:

1. class Node: — this snippet defines a node that stores data and references to its left and right children.
2. self.root = None — this snippet initializes the binary search tree as empty.
3. if root is None: return Node(value) — this snippet creates a new node when an empty position is found during insertion.
4. if value < root.data: — this snippet checks whether the value is smaller than the current node and should be inserted into the left subtree.
5. def search(self, root, value): — this snippet defines the search operation to locate a value by comparing it with the current node.
6. self.inorder(root.left) followed by print(root.data, end=" ") and self.inorder(root.right) — this snippet performs inorder traversal, displaying the BST values in ascending order.
7. self.preorder(root) and self.postorder(root) — these methods implement preorder traversal (root, left, right) and postorder traversal (left, right, root), respectively.
8. temp = self.min_value(root.right) — this snippet finds the smallest value in the right subtree when deleting a node with two children. That value replaces the deleted node, and the duplicate is then removed.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Forgot return Node(value) when root is None	Logical	The new node is not inserted into the tree	Return the newly created node.
Forgot the base condition in a traversal	Runtime — AttributeError	'NoneType' object has no attribute 'data'	Add if root: before accessing node data.
Did not handle nodes with zero or one child during deletion	Logical	The deleted node remains in the tree or its child is lost	Return the existing child, or None if there are no children.
Used the wrong replacement value when deleting a node with two children	Logical	The BST ordering becomes incorrect	Use the minimum value from the right subtree and delete its original node.

WEEK-10

AIM:

Write a python program to implement AVL trees with all its operations.

INPUT CODE:

1.py - C:/Users/Lab/Desktop/CSE25113/WEEK-10/1.py (3.11.4)

File Edit Format Run Options Window Help

```
class Node:
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
        self.height = 1
class AVLTree:
    def height(self, root):
        return root.height if root else 0

    def get_balance(self, root):
        return self.height(root.left) - self.height(root.right) if root else 0

    def right_rotate(self, y):
        x = y.left
        t = x.right
        x.right = y
        y.left = t
        y.height = 1 + max(self.height(y.left), self.height(y.right))
        x.height = 1 + max(self.height(x.left), self.height(x.right))
        return x

    def left_rotate(self, x):
        y = x.right
        t = y.left
        y.left = x
        x.right = t
        x.height = 1 + max(self.height(x.left), self.height(x.right))
        y.height = 1 + max(self.height(y.left), self.height(y.right))
        return y

    def insert(self, root, key):
        if not root:
            return Node(key)
        if key < root.key:
            root.left = self.insert(root.left, key)
        elif key > root.key:
            root.right = self.insert(root.right, key)
        else:
            return root
=        root.height = 1 + max(self.height(root.left), self.height(root.right))
        b = self.get balance(root)
```

1.py - C:/Users/Lab/Desktop/CSE25113/WEEK-10/1.py (3.11.4)

File Edit Format Run Options Window Help

```
=         root.height = 1 + max(self.height(root.left), self.height(root.right))
        b = self.get_balance(root)
        if b > 1 and key < root.left.key:
            return self.right_rotate(root)
        if b < -1 and key > root.right.key:
            return self.left_rotate(root)
        if b > 1 and key > root.left.key:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        if b < -1 and key < root.right.key:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
        return root

    def min_value_node(self, root):
        while root.left:
            root = root.left
        return root

    def delete(self, root, key):
        if not root:
            return root
        if key < root.key:
            root.left = self.delete(root.left, key)
        elif key > root.key:
            root.right = self.delete(root.right, key)
        else:
            if not root.left:
                return root.right
            elif not root.right:
                return root.left
            temp = self.min_value_node(root.right)
            root.key = temp.key
            root.right = self.delete(root.right, temp.key)
        if not root:
            return root
        root.height = 1 + max(self.height(root.left), self.height(root.right))
        b = self.get_balance(root)
        if b > 1 and self.get_balance(root.left) >= 0:
            return self.right_rotate(root)
        if b > 1 and self.get_balance(root.left) < 0:
            root.left = self.left rotate(root.left)
```

1.py - C:/Users/Lab/Desktop/CSE25113/WEEK-10/1.py (3.11.4)

File Edit Format Run Options Window Help

```
        if b > 1 and self.get_balance(root.left) < 0:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        if b < -1 and self.get_balance(root.right) <= 0:
            return self.left_rotate(root)
        if b < -1 and self.get_balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
        return root

    def search(self, root, key):
        if not root or root.key == key:
            return root
        if key < root.key:
            return self.search(root.left, key)
        return self.search(root.right, key)

    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.key, end=" ")
            self.inorder(root.right)

    def preorder(self, root):
        if root:
            print(root.key, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.key, end=" ")

    def display(self, root, level=0):
        if root:
            self.display(root.right, level + 1)
            print("    " * level + str(root.key))
            self.display(root.left, level + 1)
tree = AVLTree()
root = None
```

1.py - C:/Users/Lab/Desktop/CSE25113/WEEK-10/1.py (3.11.4)

File Edit Format Run Options Window Help

```
115
116     def display(self, root, level=0):
117         if root:
118             self.display(root.right, level + 1)
119             print("    " * level + str(root.key))
120             self.display(root.left, level + 1)
121 tree = AVLTree()
122 root = None
123 while True:
124     print("\n1.Insert\n2.Delete\n3.Search\n4.Inorder\n5.Preorder\n6.Postorder\n7.Display\n8.Exit")
125     choice = int(input("Enter choice: "))
126
127     if choice == 1:
128         key = int(input("Enter value: "))
129         root = tree.insert(root, key)
130         print("Inserted")
131     elif choice == 2:
132         key = int(input("Enter value: "))
133         if tree.search(root, key):
134             root = tree.delete(root, key)
135             print("Deleted")
136         else:
137             print("Value not found")
138     elif choice == 3:
139         key = int(input("Enter value: "))
140         print("Found" if tree.search(root, key) else "Not found")
141     elif choice == 4:
142         tree.inorder(root)
143         print()
144     elif choice == 5:
145         tree.preorder(root)
146         print()
147     elif choice == 6:
148         tree.postorder(root)
149         print()
150     elif choice == 7:
151         tree.display(root)
152     elif choice == 8:
153         break
154     else:
155         print("Invalid choice")
156
```

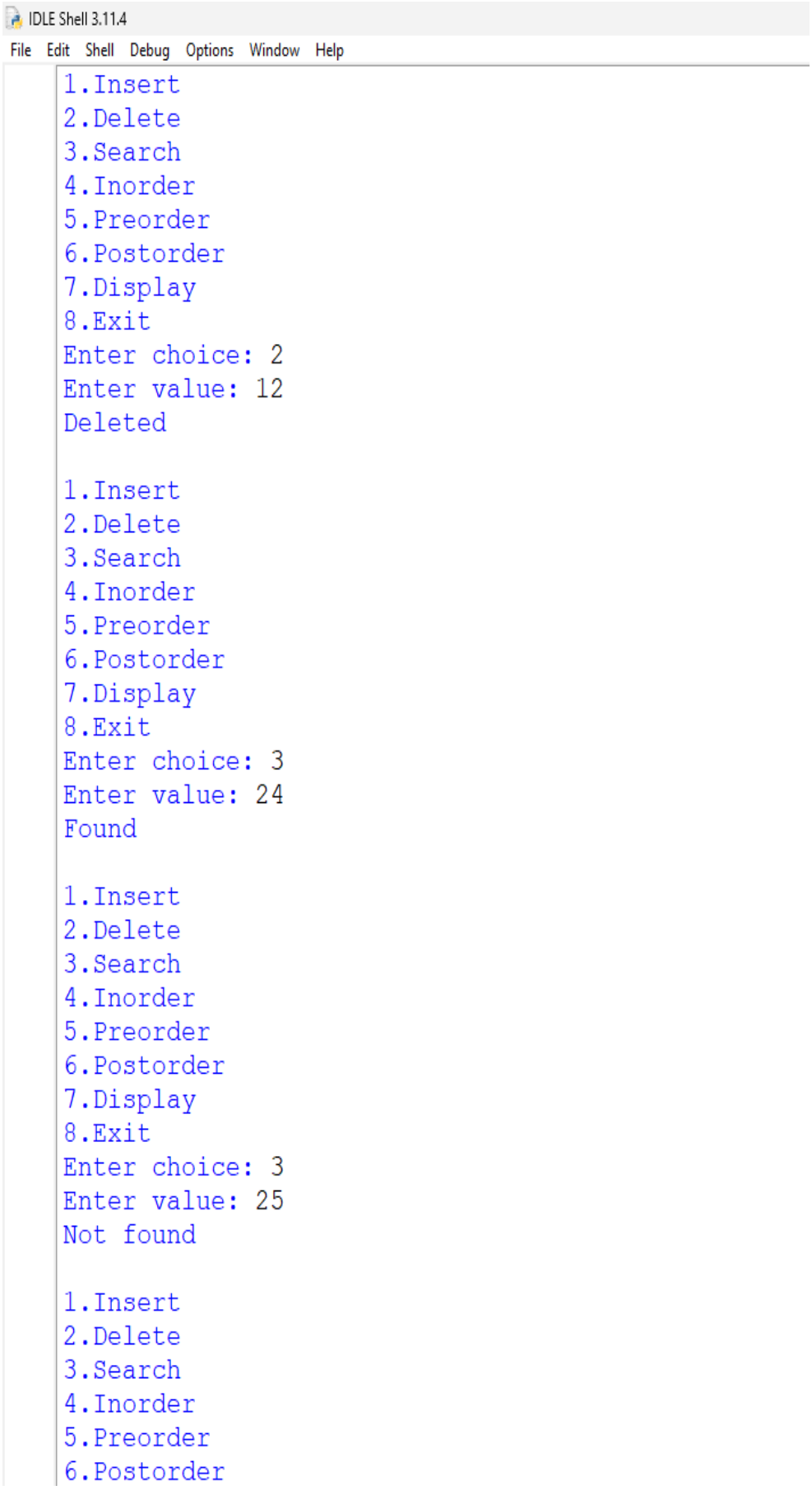
OUTPUT:

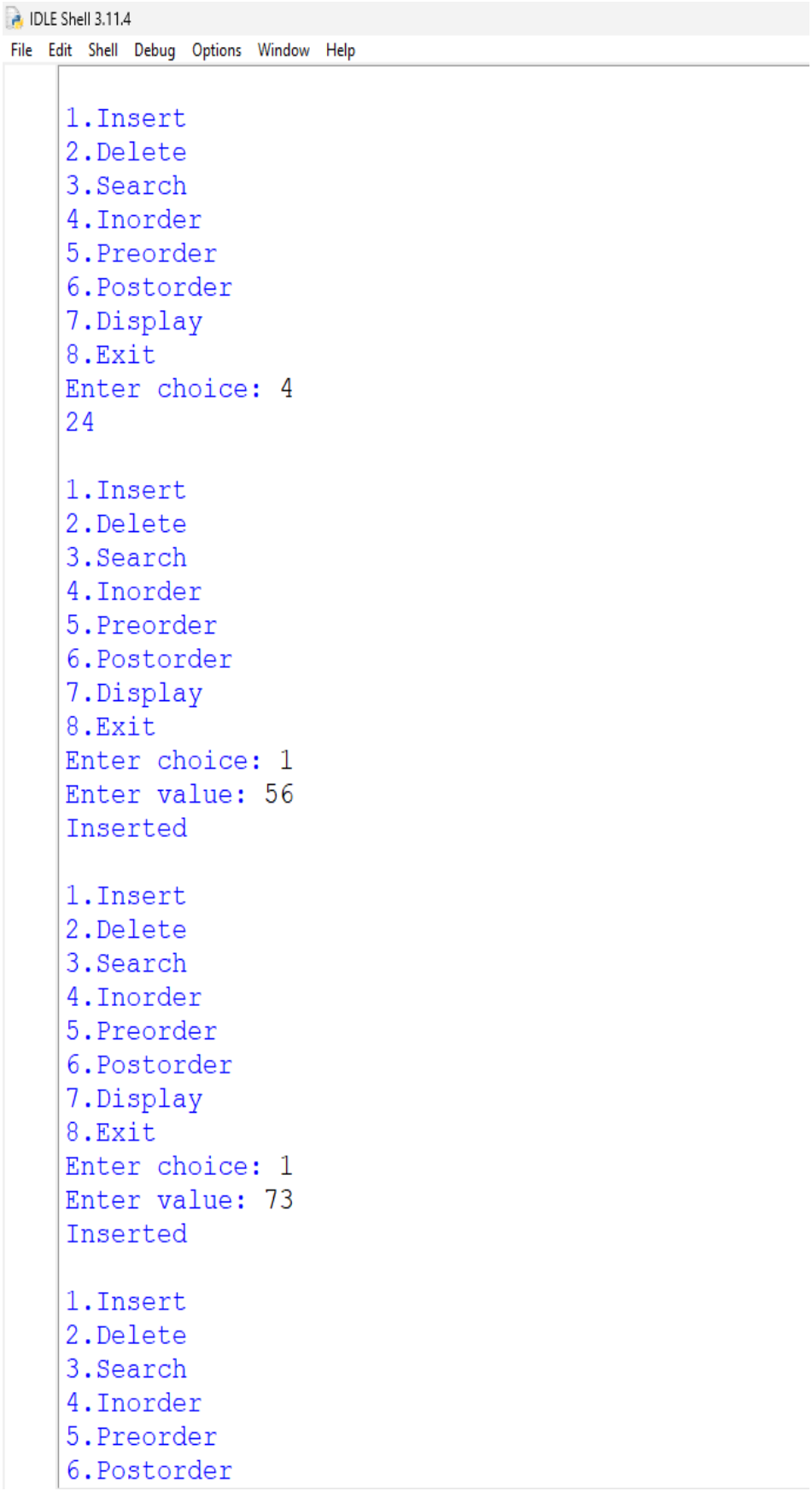
```
========== RESTART: C:/Users/Lab/Desktop/CSE25113/WEEK-10/1.py ==========

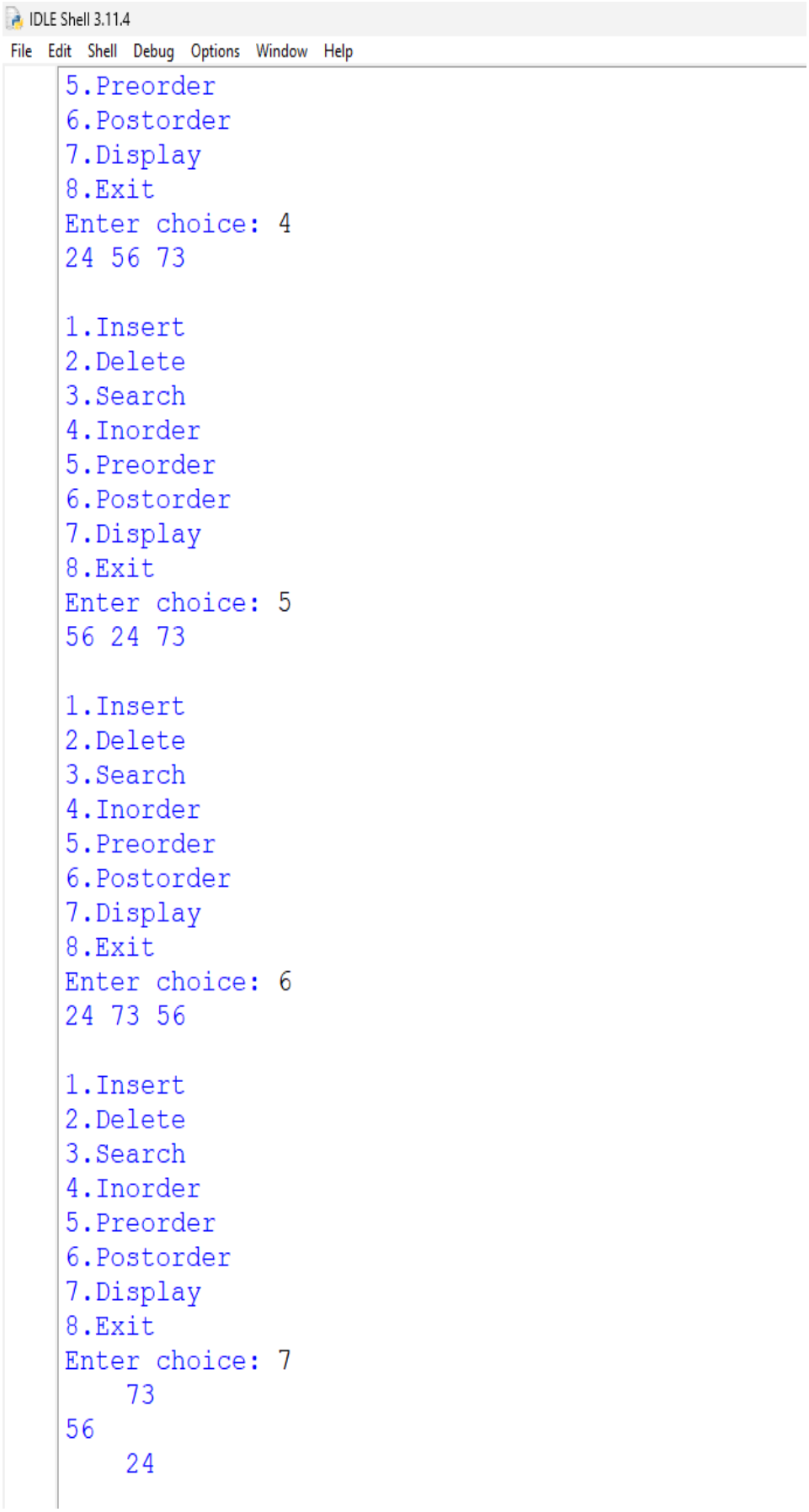
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 1
Enter value: 12
Inserted

1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 24
Invalid choice

1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 1
Enter value: 24
Inserted
```


IDLE Shell 3.11.4
File Edit Shell Debug Options Window Help
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 2
Enter value: 12
Deleted
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 3
Enter value: 24
Found
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 3
Enter value: 25
Not found
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder


IDLE Shell 3.11.4
File Edit Shell Debug Options Window Help
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 4
24
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 1
Enter value: 56
Inserted
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 1
Enter value: 73
Inserted
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder


IDLE Shell 3.11.4
File Edit Shell Debug Options Window Help
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 4
24 56 73
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 5
56 24 73
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 6
24 73 56
1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 7
73
56
24

```
8.Exit
Enter choice: 7
    73
56
    24

1.Insert
2.Delete
3.Search
4.Inorder
5.Preorder
6.Postorder
7.Display
8.Exit
Enter choice: 8
>>>
```

EXPLANATION:

1. self.height = 1 — this snippet stores the height of each node inside the node itself, starting at 1 for a new leaf. The AVL tree uses this stored value to check balance without recomputing subtree heights each time.

2. return self.height(root.left) - self.height(root.right) if root else 0 — this snippet calculates the balance factor as left height minus right height. A value above 1 means the tree is left-heavy, and a value below -1 means it is right-heavy.

3. x = y.left followed by x.right = y and y.left = t — this snippet performs a right rotation. The left child x becomes the new subtree root, y moves down to become its right child, and x's old right subtree t is reattached as y's left child. Heights are then updated for y first and x second.

4. y = x.right followed by y.left = x and x.right = t — this snippet performs a left rotation, the mirror of the right rotation. The right child y moves up, x becomes its left child, and y's old left subtree t is reattached as x's right child.

5. if b > 1 and key > root.left.key: followed by root.left = self.left_rotate(root.left) and return self.right_rotate(root) — this snippet handles the Left-Right case after insertion. The left child is rotated left first to straighten the zigzag, and then the root is rotated right. The b < -1 and key < root.right.key case is the Right-Left mirror image.

6. if b > 1 and self.get_balance(root.left) >= 0: — this snippet is the rebalancing check after deletion. The key comparison used in insertion no longer applies, so the balance factor of the child decides between a single rotation (>= 0 or <= 0) and a double rotation (< 0 or > 0).

7. temp = self.min_value_node(root.right) followed by root.key = temp.key and root.right = self.delete(root.right, temp.key) — this snippet deletes a node with two children. It copies the inorder successor (the smallest key in the right subtree) into the node, then deletes that successor from the right subtree.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Forgot to update the node height after insertion or rotation	Logical	Balance factors are calculated wrongly, so the tree becomes unbalanced or rotates when it should not	Set root.height = 1 + max(self.height(root.left), self.height(root.right)) before calling get_balance().
Forgot to return the new subtree root from a rotation	Runtime — AttributeError	AttributeError: 'NoneType' object has no attribute 'key' on the next operation	Return x from right_rotate() and y from left_rotate() so the parent links to the new root.
Used the wrong rotation for the Left-Right or Right-Left case	Logical	The tree stays unbalanced after insertion, with a zigzag shape that a single rotation cannot fix	Rotate the child first (left_rotate(root.left) or right_rotate(root.right)), then rotate the root.
Forgot to rebalance after deleting a node	Logical	The tree is still a valid BST but its height difference exceeds 1 after deletions	Update the height and check get_balance() after the delete recursion, using the child's balance to pick the rotation.
Forgot to assign the result of insert() or delete() back to root	Logical	The tree does not change, or the old root is kept after a rotation changes the top node	Use root = tree.insert(root, key) and root = tree.delete(root, key) so the new root is stored.

WEEK-11

AIM:

Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).

INPUT CODE:

1.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-11/1.py (3.14.0)

File Edit Format Run Options Window Help

```
class SeparateChaining:
    def __init__(self, size):
        self.size = size
        self.table = [[] for i in range(size)]
    def hash_function(self, key):
        return key % self.size
    def insert(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            print(key, "already exists")
            return
        self.table[index].append(key)
        print(key, "inserted at index", index)
    def search(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            print(key, "found at index", index)
        else:
            print(key, "not found")
    def delete(self, key):
        index = self.hash_function(key)
        if key in self.table[index]:
            self.table[index].remove(key)
            print(key, "deleted")
        else:
            print(key, "not found")
    def display(self):
        print("Hash table:")
        for i in range(self.size):
            print(i, "-->", self.table[i])
size = int(input("Enter size of hash table: "))
ht = SeparateChaining(size)
while True:
    print("\n--- SEPARATE CHAINING MENU ---")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
```

```
    if choice == 1:
        key = int(input("Enter key: "))
        ht.insert(key)
    elif choice == 2:
        key = int(input("Enter key to search: "))
        ht.search(key)
    elif choice == 3:
        key = int(input("Enter key to delete: "))
        ht.delete(key)
    elif choice == 4:
        ht.display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")

```

OUTPUT:

```
============ RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-11/1.py ===========
Enter size of hash table: 4

--- SEPARATE CHAINING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 1
Enter key: 12
12 inserted at index 0

--- SEPARATE CHAINING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 1
Enter key: 13
13 inserted at index 1

--- SEPARATE CHAINING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 1
Enter key: 14
14 inserted at index 2
```

```
--- SEPARATE CHAINING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 2
Enter key to search: 12
12 found at index 0

--- SEPARATE CHAINING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 3
Enter key to delete: 14
14 deleted

--- SEPARATE CHAINING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 5
Program terminated
>>>
```

EXPLANATION:

1. self.table = [[] for i in range(size)] — this snippet creates the hash table, where every index has its own separate empty list called a chain.

2. return key % self.size — this snippet is the hash function. It turns a key into an index between 0 and size - 1 using the remainder.

3. index = self.hash_function(key) — this snippet works out which index (bucket) the key belongs to before any operation.

4. if key in self.table[index]: print(key, "already exists") — this snippet stops the same key from being stored twice in the table.

5. self.table[index].append(key) — this snippet handles a collision by adding the new key to the end of the chain already at that index.

6. if key in self.table[index]: print(key, "found at index", index) — this snippet searches only the one chain at the hashed index, not the whole table.

7. self.table[index].remove(key) — this snippet removes the key from its chain, and the other keys in the same chain stay where they are.

8. print(i, "-->", self.table[i]) — this snippet prints every index along with the list of keys chained at that index.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Created the table as [[]] * size	Logical	The same key appears at every index of the table	Use [[] for i in range(size)] so each index gets its own list
Called remove(key) without checking if the key is present	Runtime — ValueError	ValueError: list.remove(x): x not in list	Check if key in self.table[index]: before removing
Entered 0 as the size of the hash table	Runtime — ZeroDivisionError	ZeroDivisionError: integer modulo by zero	Enter a table size greater than 0

INPUT CODE:

1.2.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-11/1.2.py (3.14.0)

File Edit Format Run Options Window Help

```
class LinearProbing:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size
    def hash_function(self, key):
        return key % self.size
    def insert(self, key):
        index = self.hash_function(key)
        i = 0
        while i < self.size:
            pos = (index + i) % self.size
            if self.table[pos] is None or self.table[pos] == "DELETED":
                self.table[pos] = key
                print(key, "inserted at index", pos)
                return
            i += 1
        print("Hash table is full")
    def search(self, key):
        index = self.hash_function(key)
        i = 0
        while i < self.size:
            pos = (index + i) % self.size
            if self.table[pos] is None:
                return -1
            if self.table[pos] == key:
                return pos
            i += 1
        return -1
    def delete(self, key):
        pos = self.search(key)
        if pos == -1:
            print(key, "not found")
        else:
            self.table[pos] = "DELETED"
            print(key, "deleted")
    def display(self):
        print("Hash table:")
        for i in range(self.size):
            if self.table[i] is None or self.table[i] == "DELETED":
                print(i, "--> Empty")
```

```
                print(i, "--> Empty")
            else:
                print(i, "-->", self.table[i])
size = int(input("Enter size of hash table: "))
ht = LinearProbing(size)
while True:
    print("\n--- LINEAR PROBING MENU ---")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        key = int(input("Enter key: "))
        ht.insert(key)
    elif choice == 2:
        key = int(input("Enter key to search: "))
        pos = ht.search(key)
        if pos == -1:
            print(key, "not found")
        else:
            print(key, "found at index", pos)
    elif choice == 3:
        key = int(input("Enter key to delete: "))
        ht.delete(key)
    elif choice == 4:
        ht.display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")

```

OUTPUT:

```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-11/1.2.py
Enter size of hash table: 3

--- LINEAR PROBING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 1
Enter key: 12
12 inserted at index 0

--- LINEAR PROBING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 2
Enter key to search: 12
12 found at index 0

--- LINEAR PROBING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 2
Enter key to search: 3
3 not found
```

```
--- LINEAR PROBING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 3
Enter key to delete: 12
12 deleted

--- LINEAR PROBING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 4
Hash table:
0 --> Empty
1 --> Empty
2 --> Empty

--- LINEAR PROBING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 5
Program terminated
>>>
```

EXPLANATION:

1. self.table = [None] * size — this snippet creates a table where every slot holds only one key. None marks a slot that has never been used.

2. pos = (index + i) % self.size — this snippet checks the next slot one step at a time when a collision happens, and wraps back to index 0 after the last slot.

3. if self.table[pos] is None or self.table[pos] == "DELETED": — this snippet accepts both never-used slots and slots freed by deletion as places to store the key.

4. while i < self.size: — this snippet limits probing to at most size slots, so the loop always ends.

5. print("Hash table is full") — this snippet runs only when every slot has been checked and none of them is free.

6. if self.table[pos] is None: return -1 — this snippet stops the search at a never-used slot, because the key could not have been placed beyond it.

7. if self.table[pos] == key: return pos — this snippet returns the slot number where the key was found.

8. pos = self.search(key) — this snippet reuses the search function inside delete to find the key's slot.

9. self.table[pos] = "DELETED" — this snippet marks the slot as deleted instead of emptying it, so the search can still continue past this slot to find keys stored after it.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Set the deleted slot to None instead of "DELETED"	Logical	A key shows "not found" even though it is present in the table	Mark deleted slots as "DELETED" so the search continues past them
Did not allow "DELETED" slots in the insert condition	Logical	Hash table is full (shown even after keys are deleted)	Use if self.table[pos] is None or self.table[pos] == "DELETED":
Wrote (index + 1) instead of (index + i)	Logical	A key is stored one slot away from its hash index, and keys that collide show "Hash table is full"	Use pos = (index + i) % self.size

INPUT CODE:

1.3.py - C:/Users/vishn/Desktop/DSA PYTHON/WEEK-11/1.3.py (3.14.0)

File Edit Format Run Options Window Help

```
class QuadraticProbing:
    def __init__(self, size):
        self.size = size
        self.table = [None] * size
    def hash_function(self, key):
        return key % self.size
    def insert(self, key):
        index = self.hash_function(key)
        i = 0
        while i < self.size:
            pos = (index + i * i) % self.size
            if self.table[pos] is None or self.table[pos] == "DELETED":
                self.table[pos] = key
                print(key, "inserted at index", pos)
                return
            i += 1
        print("No free slot found for", key)
    def search(self, key):
        index = self.hash_function(key)
        i = 0
        while i < self.size:
            pos = (index + i * i) % self.size
            if self.table[pos] is None:
                return -1
            if self.table[pos] == key:
                return pos
            i += 1
        return -1
    def delete(self, key):
        pos = self.search(key)
        if pos == -1:
            print(key, "not found")
        else:
            self.table[pos] = "DELETED"
            print(key, "deleted")
    def display(self):
        print("Hash table:")
        for i in range(self.size):
            if self.table[i] is None or self.table[i] == "DELETED":
                print(i, "--> Empty")
```

```
            else:
                print(i, "-->", self.table[i])
size = int(input("Enter size of hash table: "))
ht = QuadraticProbing(size)
while True:
    print("\n--- QUADRATIC PROBING MENU ---")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Display")
    print("5. Exit")
    choice = int(input("Enter your choice: "))
    if choice == 1:
        key = int(input("Enter key: "))
        ht.insert(key)
    elif choice == 2:
        key = int(input("Enter key to search: "))
        pos = ht.search(key)
        if pos == -1:
            print(key, "not found")
        else:
            print(key, "found at index", pos)
    elif choice == 3:
        key = int(input("Enter key to delete: "))
        ht.delete(key)
    elif choice == 4:
        ht.display()
    elif choice == 5:
        print("Program terminated")
        break
    else:
        print("Invalid choice")

```

OUTPUT:

```
========== RESTART: C:/Users/vishn/Desktop/DSA PYTHON/WEEK-11/1.3.py ==========
Enter size of hash table: 3

--- QUADRATIC PROBING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 1
Enter key: 56
56 inserted at index 2

--- QUADRATIC PROBING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 1
Enter key: 57
57 inserted at index 0

--- QUADRATIC PROBING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 2
Enter key to search: 5
5 not found
```

```
--- QUADRATIC PROBING MENU ---
1. Insert
2. Search
63. Delete
4. Display
5. Exit
Enter your choice: 3
Enter key to delete: 56
56 deleted

--- QUADRATIC PROBING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 4
Hash table:
0 --> 57
1 --> Empty
2 --> Empty

--- QUADRATIC PROBING MENU ---
1. Insert
2. Search
3. Delete
4. Display
5. Exit
Enter your choice: 5
Program terminated
>>>
```

EXPLANATION:

1. pos = (index + i * i) % self.size — this snippet jumps 1, 4, 9, 16… slots away from the hashed index on each collision, instead of moving one slot at a time.

2. i = 0 — this snippet makes the first probe land on the hashed index itself, because 0 × 0 = 0.

3. i += 1 — this snippet increases the probe number, so the gap between checked slots keeps growing (1, 3, 5…). This spreads colliding keys apart and reduces clustering.

4. print("No free slot found for", key) — this snippet reports that the key could not be placed. Quadratic probing can miss empty slots, so this can happen even when the table is not completely full.

5. while i < self.size: (in search) — this snippet makes search follow the same i × i jumps as insert, so it checks the slots in the same order the key was placed.

6. if pos == -1: print(key, "not found") — this snippet uses the return value -1 from search as a signal that the key is not in the table.

7. if self.table[i] is None or self.table[i] == "DELETED": print(i, "--> Empty") — this snippet shows never-used slots and deleted slots as Empty in the display.

ERROR TABLE:

ERROR	TYPE	DISPLAYED AS	FIX
Wrote i * 2 instead of i * i in the probe formula	Logical	Keys are placed at the wrong positions, for example 17 at index 5 instead of 4	Use pos = (index + i * i) % self.size
Used a non-prime table size such as 10	Logical / handled	No free slot found for 65 (shown even though empty slots are left)	Use a prime table size and keep the table less than half full
Forgot the return after inserting the key	Logical	The same key is stored in several slots and "No free slot found" is also printed	Add return right after storing the key
Created the table as (None,) * size	Runtime — TypeError	TypeError: 'tuple' object does not support item assignment	Use a list: self.table = [None] * size